

Generation of Sheet Music Notation for Piano Using Deep Learning

A dissertation submitted in partial fulfilment of

the requirements for the degree of

BACHELOR OF *SCIENCE* in Computer Science

in

The Queen's University of Belfast

by

Adam McWilliams

April 2023

SCHOOL OF ELECTRONICS, ELECTRICAL ENGINEERING and COMPUTER SCIENCE

CSC3002 – COMPUTER SCIENCE PROJECT

Dissertation Cover Sheet

A signed and completed cover sheet must accompany the submission of the Software Engineering dissertation submitted for assessment.

Work submitted without a cover sheet will **NOT** be marked.

Student Name:	Adam McWilliams	Student Number:	40266924
Project Title:	Generation of Sheet Music Notation for Piano Using Deep Learning		
Supervisor:	DR. Reza Rafiee		

Declaration of Academic Integrity

Before submitting your dissertation please check that the submission:

1. Has a full bibliography attached laid out according to the guidelines specified in the Student Project Handbook
2. Contains full acknowledgement of all secondary sources used (paper-based and electronic)
3. Does not exceed the specified page limit
4. Is clearly presented and proof-read
5. Is submitted on, or before, the specified or agreed due date. Late submissions will only be accepted in exceptional circumstances or where a deferment has been granted in advance.

   By submitting your dissertation you declare that you have completed the tutorial on plagiarism at http://www.qub.ac.uk/cite2write/introduction5.html and are aware that it is an academic offence to plagiarise. You declare that the submission is your own original work. No part of it has been submitted for any other assignment and you have acknowledged all written and electronic sources used.
6. If selected as an exemplar, I agree to allow my dissertation to be used as a sample for future students. (Please delete this if you do not agree.)



Adam McWilliams *Date of submission* 24/04/2023

Acknowledgements

I would like to specifically thank individuals who helped me throughout the creation of this document.

My supervisor, DR. Reza Rafiee, for his knowledge and insight into every aspect of audio processing in machine learning. His guidance was invaluable in helping to direct my research and development, as well as giving me the freedom to pursue my personal areas of interest within this field.

My parents, Mr. Simon & Mrs. Heather McWilliams, for their unwavering support and encouragement throughout this entire process. I would like to especially note the support my mother gave me through my research while she was undergoing breast cancer surgery and how valuable that was in overcoming any challenges in this project.

My proof-reader, Miss Amy McWilliams, for her insight and overview into the development of this document over various iterations.

Abstract

This project researches and explores the possibilities of using deep leaning to interpret digital audio recordings. The goal of the research is to create a solution which is able to read and interpret music similar to a human. From this the system then attempts to replicate what it has heard through musical engraving, or the process of turning musical ideas into notation so that they can be replicated. There are several steps involved in the interpretating of music. In the pre-processing of data to ensure compatibility mathematical techniques are used to convert these musical recordings into images. These images can then be read and understood by a convolutional neural network. This solution aims to provide access to this technology to all people regardless of their experience with artificial intelligence systems. Through user friendly design and easy to understand implementation.

Contents

1.0 Introduction 1
1.1 Problem Background 1
Musical Notation 1
Problem Description 2
Theory of digital Audio 3
Theory of deep learning 3
1.2 Potential areas of concern and mitigations 4
Analogous Data 5
Deep Learning Unpredictability's 6
Polyphonality 6
2.0 System Requirements and Specification 7
2.1 Dual solution Approach 7
2.2 Model Training 7
Dataset Analysis 7
Data Pre-processing 8
Model Generation 9
Model Validation 10
2.3 User-based system Pipeline 11
Web Interface 11
Audio Pre-processing 13
Inference 14
Interpretation of Results 14
Midi Generation 15
Score Generation 16
3.0 Design 16
3.1 Architectural Design 16
From Raw Data to a Batch of Spectrograms 17
Fitting data to the model 17
Website Interface 18
Application Call 19
Object Instantiation 19
Inference 21
3.2 User Interface Design 22

Home Page 22
User-Guide 25
Final Designs 25
Home Page 26
About Page 27
User-Guide 28
3.3 Component Design 29
Model Loading 29
Pre-processing 29
Inference and Midi Generation 30
Score Engraving 33
3.4 Model Design 33
Data Pre-processing 33
Model Training 34
4.0 Implementation 37
4.1 Python for implementation 37
4.2 Featured packages and libraries 38
4.3 Model Training in Jupyter Notebook 38
4.4 Key features of Musical Notation Application 42
Important functional elements 42
5.0 Testing 44
5.1 Unit Testing 44
Resolutions 44
5.2 Manual Testing 45
5.3 Model Testing 46
5.4 Integration Testing 46
6.0 Evaluation 46
6.1 Final Evaluation 46
Model Evaluation 47
Inference Evaluation 47
Integration Evaluation 47
7.0 Conclusion 48

1.0 Introduction

The purpose of this paper is to explain the development and findings of the musical notation generator application created. The bibliography contains a link to the repository containing the code for this solution. This solution was developed as two overarching paths, developing and training of the deep learning network to make predictions on musical audio data, and an inference solution which uses this model to generate predictions and structure them into a piece of musical notation.

1.1 Problem Background

Musical Notation

Performing music is a pastime that has been enjoyed for millennia, it has changed and evolved over time into something which is enjoyed and is widely accessible. The idea of writing down music in its own notation as a series of instructions to allow a musician to recreate it accurately has been around as early as the Babylonian age. (Kilmer & Civil, 1986) Since it was first written down, music notation has transformed greatly to simplify how to perform a specific piece. However, one thing that appears to have remained constant throughout the history of written music is the fundamental idea of note-based transcription.

In a modern manuscript there are many different components working together. These can be as simple as a bar containing few notes, to more complex things such as ornaments, dynamic markings and articulations.



Figure 1.1 An example of a music staff

Taking the image above from left to right, we have certain symbols each with very specific meanings. Initially, we have a treble clef to denote what range of pitches we are using. Next, we have the time signature which informs the musician of the rhythmic pattern in the music, meaning how many beats are to be played in each bar, as well as how the musician should ‘count’ the music. Now that the initial notation has been read and understood, the score now presents us with the instructions on how to play this piece.

We have three notes all at different pitches. We can read the exact pitch of the note based on its vertical position. These notes also contain information on how long each note should be played for, this is known as a note value and the value of a note changes based on the shape

of the symbol. Finally, we have a bar line to inform the musician that the count has started again and as there is another note immediately after the bar line, it is the ‘first’ note again. In this example the bar line appears in that position as two notes with a quarter value and one note with a half value have appeared, which is the equivalent of four quarter notes the same as our time signature.

Musical notation also typically includes a key signature when the music is being performed in a key other than `C`, this tells the player that when a note appears with a certain pitch, if that note has ♯ (sharp) in its key signature, it should be played one semitone higher. However, if a note has ♭ (flat) it should be performed one semitone lower. A key signature is technically not required in musical notation, however, it greatly assists a performer when they read the score. These are the basic rules followed when generating sheet music notation in this system.

Problem Description

Typically, when we transcribe music, it is so that is can be interpreted from the page into a performance. This project however provides musicians with the opposite approach to the norm. This system at a high level accepts a digital recording of a performance and generates a document of sheet music using an artificial intelligence deep learning model.

In recent times, the popular music of an era has moved away from styles which would typically be composed and transcribed first and then performed live later by a person or people who are not the composer or may not even have a relationship with the composer at all. This has created a decline in both the production of music notation and a decrease in musical literacy. (Henschen, 2018)

This has had serval effects on modern music. The music industry was built upon the exchange of musical ideas through commercially making scores available for a performer to recite. With the exchange of musical ideas instead of music, performers were free to improvise or modify the works they performed creating a more unique aspect to each performance. Contrastingly, in present times the music industry is based on selling specific recordings of pieces causing a decline in the variety of music available due to fewer interpretations of pieces with improvisations.

This highlights the issue that as sheet music becomes less available, the diversity in our music suffers. Therefore, the solution that I have developed, allows for more people to accurately recreate recordings of popular music advancing their skills and musicianship more

effectively. This musical notation generation solution will not only benefit musicians, but also those who enjoy music, which comprises 90% of the global population. (Nielsen, 2015)

A major benefit of this solution is the ability for anyone to record and catalogue their musical ideas. This makes it easier to keep track of original ideas and share those ideas with others. This is very relevant today as musical intellectual property and copyright law are more prolific, particularly given the recent high-profile cases of the proposed theft of musical ideas from one popular song to another. The solutions developed in this project allows a musician to be able to prove that a musical motif belongs to them as sheet music can be compared much easier than recordings in these legal cases.

Theory of digital Audio

The input for this system is a digital audio recording. This can be in one of various formats, the most common being .wav, .mp3, and .aac. Digital audio files are a binary representation of sound frequencies over time. A single note or any sound is the result of vibrations in the air and the speed at which these vibrations occur is known as the frequency.

The frequency of a sound wave determines its pitch, and for musical notes, these frequencies oscillate at regular intervals. These pitches can be measured, for example a microphone, measures the air pressure thousands of times per second which then recreates the sound by moving the air in the same way through a speaker. When we visualise these values in series, we generate a waveform, which can be used to analyse the sound recording further as the height of the graph is the volume and the gradient is the pitch. This is used by the program to generate data which can be understood by a neural network.

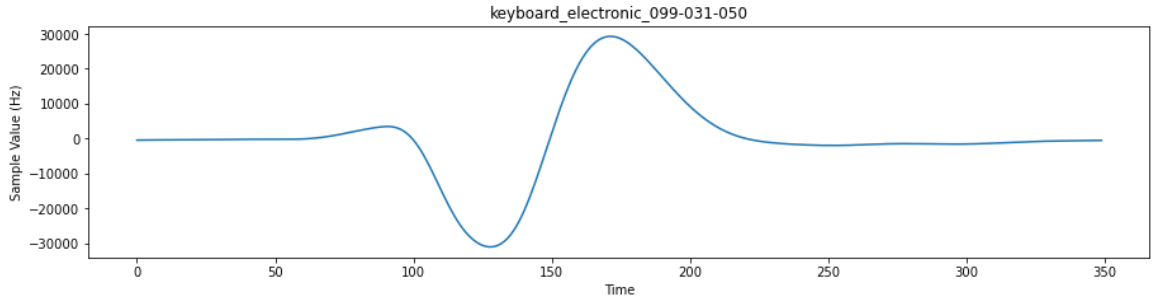


Figure 1.2 Part of a waveform for an electronic keyboard playing $D^{\#7}$ at 50% volume.

Theory of deep learning

This project is based around using a deep learning artificial intelligence model to make predictions about the music it has been given. A deep learning model is a type of neural

network. An artificial neural network can be thought of as a circuit or a method of connecting together various nodes. Each of these nodes is modelled after a biological neuron which can be found inside of the brains of animals. (Yang & Yang, 2014)

The theory behind this is a method of classifying data which cannot be solved traditionally. Each node has a weight associated with it, which determines how great its impact is on the rest of the network. Each node is activated differently based on the values on the first layer to create associates, similar to how different neurons connect to each other in a human brain, by changing this weight. In a simple neural network, the nodes in the input layer are connected directly to the output of nodes. The difference between this and a deep learning model is the introduction if hidden layers which connect the input and output layers. There are different types of deep learning networks which are compared in a later section, this will explain the network used in this project.

The following is a visualisation of how a deep learning network is connected.

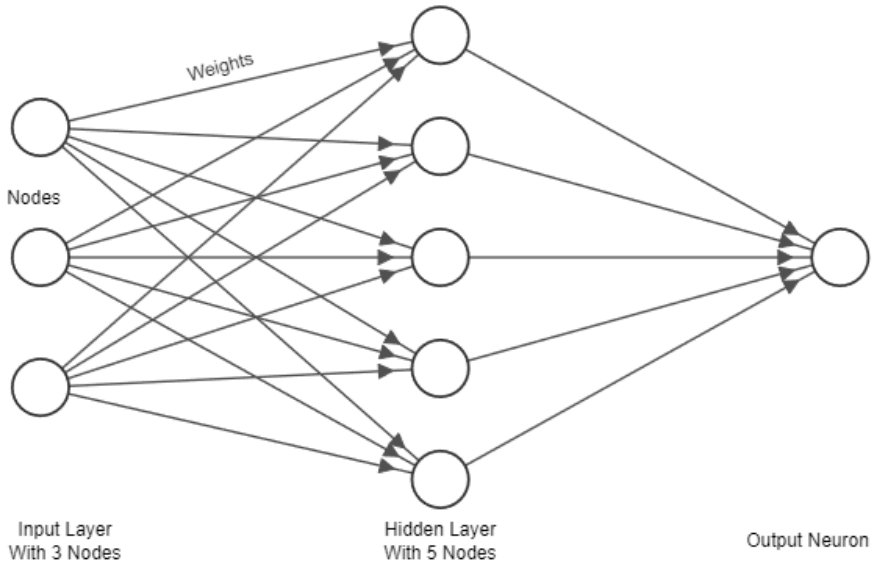


Figure 1.3 A simple deep learning network with a single hidden layer

1.2 Potential areas of concern and mitigations

When developing a deep learning solution such as this, there are many things to be considered because of the problems associated with deep learning neural networks. These potential issues often revolve around the data that is being prepared for training or how the data is fed into the input layer of the network.

Analogous Data

For this project, the data used for training and validation of the model was imported from the NSynth dataset which allows for the model to learn on over 300,000 different audio samples. (Engel, et al., 2017) This is far more than a single person would be able to collect. This dataset contains samples with over one hundred different pitches, over a dozen different instruments (including these instruments with different timbres), which describes how the same instrument can sound different. This is tremendous for the artificial intelligence as these different instruments can play a note at the same pitch. Though, producing drastically different waveforms.

However, this dataset also has an issue where each of these samples are all recorded in the same format of a single note playing for 4 seconds. This should not create any major flaws in the deep learning algorithm. However, it also drastically decreases the probability that the model can make context-based predictions. Therefore, without context-based learning, in this project predictions are made at an individual note level. The application trims the audio file to fixed length inputs for the model, as opposed to making predictions on larger musical phrases which are much more complex.

Dynamics, Articulations, and ornaments

When humans listen to and interpret music, the brain makes context through relating each note together. This is not possible with a machine learning solution being trained on individual notes. The artificial brain has a much more difficult task of processing music as it will predict things that would not make sense to a human. For this reason, the scope of this project has been reduced to exclude non-essential elements of musical notation.

In scores composed by humans there are often symbols which convey much more advanced musical techniques such as ornaments or articulations. These are used to instruct a performer in what style they must play their instrument. A neural network does not understand the concept of an instrument, nor the complexity of how to play one in different styles. This is an advantage however, as with the context-free approach the model only needs to understand the pitch of a note and not the emotional meaning behind it.

Overfitting

In training machine learning models, the concept of generalisation is very important. This means that a model is able to accurately make predictions based on an input in many different variations. Overfitting is when a network generalises very poorly but is still able to

make predictions very accurately on data it has been trained on. When working with audio data the risk of overfitting is significantly increased due to the number of data associated with a relatively small sample as well as how much variance there is in the recording process. All of this random data in a sample is known as noise, therefore for a model to accurately make predictions it must be pre-processed down to a much simpler form.

Deep Learning Unpredictability's

It is the nature of machine learning that training a neural network is unpredictable. This is because there are often hundreds of thousands or even millions of different variables which are instantiated randomly. This means that every time a dataset is fit to a model during training the accuracy of the model cannot be determined beforehand. This means that when designing and training a model to perform a specific task there will likely be many different iterations with the performance improving gradually over time. Alongside this training each model is also very computationally expensive and takes a very long time to train. This is a great area of concern when designing deep learning models as there is a trade-off to consider between performance and the time available to train. To reach a more optimum solution for the deep learning portion of this project, it would make sense to keep the network design relatively simple as more complex models may be able to make better predictions, but they will become very resource intensive and take too much time to be useful.

Polyphonality

A simple music note will follow a sine wave pattern. Whenever multiple notes are played together the resulting sound is the sum of each of these waves. This is known as polyphony where a new sound is created by combining the base notes. This is important as the same effect occurs with a single note due to its overtones. This is when a note is played on a real instrument and the sound wave vibrates at the frequency of the note, known as the fundamental frequency, as well as related higher frequencies. This can be an issue when classifying an audio sample to a specific pitch as there are 88 notes available on a standard piano, but the number of permutations of those notes is a number with over 130 digits. It is impossible to train a model on all of these combinations therefore pre-processing is performed on each sample to extract the fundamental frequency from it.

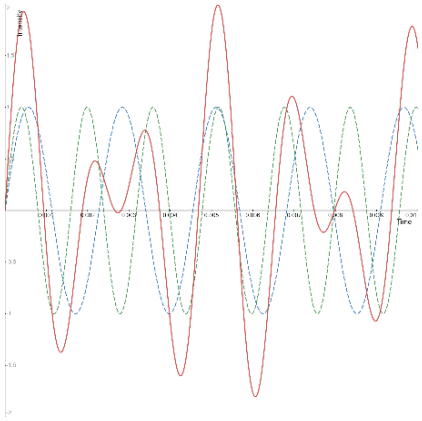


Figure 1.4 A series of Blue=440Hz, Green=628Hz and their composite function in red

2.0 System Requirements and Specification

2.1 Dual solution Approach

The music notation generator app in reality is the result of various separate solutions which have been developed working together to generate the desired output for the user. The main two development areas for this project are the creation and training of a suitable deep learning model and a solution to enable a user access to the model to make predictions. This modular approach allows for the model to be entirely independent from the web server that it is hosted on. If a new model is required, only a saved model file is needed with no changes to the code.

2.2 Model Training

To develop a deep learning neural network, it must first be designed. Then the model is compiled and trained against the full dataset, adjusting the weights so that the model can make predictions more accurately. Before a model can be designed, the data that is going through the network must be analysed and fully understood.

Dataset Analysis

To process the data used in the model training it must be annotated with a label to tell the network information about a sample and the data must be a fixed length. The NSynth dataset exists as three directories of training, testing and validation .wav files which do not overlap. These splits can be used to increase the model's performance through self-validation. Initially this data should be loaded into python so that it can be represented as a NumPy array of integers before visualising them. By plotting waveforms with Matplotlib you are able to see that each sample has 64,000 data points, however a large portion of the samples only

contain useful data at the start. This allows the amount of data that the model must process to decrease substantially as after the 10 thousandth value the rest of the sample values approach zero.

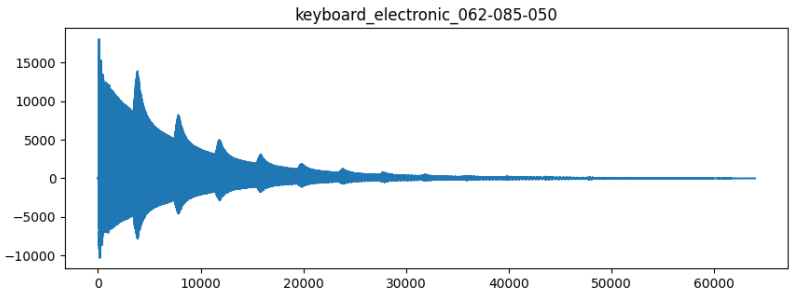


Figure 2.1 A Waveform of an electronic keyboard playing C#6

After establishing a fixed length for the data, it needs to be annotated. Each wave file in the dataset has information about it in its filename. The first word in the filename states the instrument in the recording, next is if the sample was recorded or digitally synthesized. The tag acoustic tells that the sample was recorded, and the electronic tag shows that a sample was digitally created. Finally in the filename are three numbers, these are a number to define the instrument as an integer value, the pitch of the sample using scientific pitch notation and the volume of the sample as a percentage. All of this information is used to generate a label dictionary which can be referred to during the next stage of data pre-processing.

Data Pre-processing

At this stage the dataset has been annotated and can be represented as a collection of NumPy arrays each of length 10,000. This has not solved the issues addressed in the previous section to do with polytonality or the issue of overtones. To solve this, we can use a transformation on the waveform to generate a spectrogram. To separate a sound wave into its composite frequencies a Fourier transform is used. The mathematics behind a Fourier transform is:

$$X(\tau, \omega) = \int_{-\infty}^{\infty} x(t)w(t-\tau)e^{i\omega t}\,dt$$

$$Where:$$

$$x = input\ signal$$

$$\omega = frequency$$

$$\tau = time$$

$$W(x) = window\ function$$

In simpler terms this function scans through the array of values from the audio file and extracts information about the component frequencies. This can then be interpreted as which frequencies are present over time. The output size of this function is the same as the number of these comparisons which are made, this is calculated by:

$$N = \frac{(len(x) - 1) + 2(i)}{i}$$

$$Where:$$

$$Len(x) \ = \ size\ of\ input\ signal$$

$$i \ = \ frame\ step\ in\ window\ function$$

$$N \ = \ output\ size$$

Therefore, the input layer of the network will have N nodes. For this dataset where the samples are 10,000 values long a frame step size of 64 would produce an output of size 158. This number is different in practice due to differences in implementations. After transforming the dataset, the audio files are now represented as spectrogram images. This is of huge benefit as it allows for the neural network to utilise many benefits that come with image processing such as convolutions.

Model Generation

After converting audio files into spectrograms, each sample is now represented by a fixed size image. The shape of these images determines the number of neurons in the first layer of the network. After deciding the size of the input layer, the next import stage is to establish the size of the output layer.

The purpose of this deep learning model is to make predictions on what note is being played given an audio input. This is known as a classification problem, to solve this type of problem each neuron in the output layer should represent a different class. A standard already exists for digitally storing music known as midi, as a part of the midi standard also exists scientific pitch notation. This notation is used to distinguish between 128 different pitch values as such:

Octave / Note	0	1	2	3	4	5	6	7	8
C	12	24	36	48	60	72	84	96	108
C♯/D♭	13	25	37	49	61	73	85	97	109
D	14	26	38	50	62	74	86	98	110
D♯/E♭	15	27	39	51	63	75	87	99	111

E♭/D♯	16	28	40	52	64	76	88	100	112
F	17	29	41	53	65	77	89	101	113
F♯/G♭	18	30	42	54	66	78	90	102	114
G	19	31	43	55	67	79	91	103	115
G♯/A♭	20	32	44	56	68	80	92	104	116
A	21	33	45	57	69	81	93	105	117
A♯/B♭	22	34	46	58	70	82	94	106	118
B	23	35	47	59	71	83	95	107	119

Figure 2.2 The table of midi note values

This means that the model will predict the pitch of a note by activating the corresponding neuron. The remaining thing to consider then is the hidden layers within the network. The model is built in python using the Keras library which is built on the TensorFlow 2.x framework. (TensorFlow, 2015) This gives the model access to many different types of layers to choose from in its design. These layers will be discussed more in depth in the design chapter.

Model Validation

After the model is design it needs to be trained and the validated across different datasets to measure its performance metrics. The two most important metrics for measuring how good a model is, are accuracy and loss. The accuracy is calculated by:

$$Acc\% = \frac{Correct\ Predictions}{Total\ Predictions} \cdot 100$$

A model which is learning correctly should start with a low accuracy and approach 100% after a number of iterations. An accuracy of 100% is impossible therefore when training a network, this means that when to stop the training is up to the developer's decision.

The accuracy of the model is calculated in three different ways: twice during training as well as after training is completed. The first method for calculating accuracy is using the training data. This occurs every single time that data is fed into the network and is based on the size of a single batch from the training dataset of 289,205 samples. After the network has seen all of the different samples a full epoch has passed. This is where the model will calculate its accuracy on data it has not seen before to test if it is truly learning the general patterns in the data samples, or if it is overfitting by memorising the training dataset. To calculate this, the validation dataset of 12,678 new samples is passed through the network. This accuracy value

is far more important than the training accuracy. This validation accuracy is then used to decide if it is worthwhile continuing training or if the process must be restarted.

The final way that a model's accuracy is measured is by using the testing dataset of 4,096 samples and manually reading the predictions of the model after it has finished training. This testing accuracy is how well the model actually performs, and therefore is required to be as high as possible. For this project an accuracy of over 90% is expected. This is suitable as this network should then be able to correctly predict most musical phrases, which are usually 4 bars long with only a single mistake.

A developer can utilise the loss function metrics during training in addition to the accuracy metrics to determine the performance of the model. The loss of a machine learning model is the difference between the predicted value and the actual value of a sample. Ideally the loss function of a model will decrease after each iteration as the accuracy increases. The model is considered to have finished learning when the loss starts to increase or remains constant, it is at this point where the training will cease. Ideally the model will have a training and validation accuracy of over 90% when this occurs.

2.3 User-based system Pipeline

The remainder of this projects development then is focused around using the model to make predictions by musicians in a suitable way. This involves creating a friendly user-interface to receive a musical recording to analyse. The system will then need to parse this piece into smaller parts to pass through to the model and the system will then need to interpret the predictions made by the model.

Web Interface

The first thing to design, should be a way for a user to interact with the system. There are many different options available, such as a desktop application, a python graphical user-interface built using a library such as Tkinter, or a web application hosted on a python server. This project uses a web application as it brings many benefits.

Computation with any deep learning model is complicated and may require very powerful hardware not available to the users. This problem can be avoided by utilising the processing powers of the server's hardware instead. Another advantage of using a web-based model is that the project is centrally located so that any updates or changes that are needed to be pushed can all be managed from the host machine rather than waiting for users to update their

local versions. A final major benefit of using a web-based solution is that it does not require the users to install any new software onto their device the only prior knowledge required is how to use a web browser.

The web application for the musical notation generator consists of three pages, a home page for the main functionality of the application, an information page and a user guide. The home page of the application has a few basic requirements as this is how the model will get all of the data it requires to generate sheet music. The page should consist of a navigation bar, an entry field for the user to upload a recording and a form for the user to enter information about the score. The following is just one potential design:

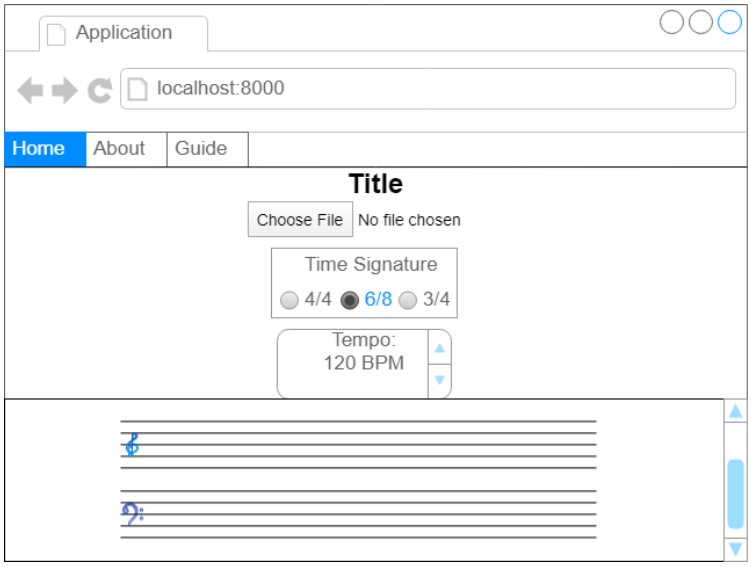


Figure 2.3 A mockup design for the homepage for the web interface

The web application also holds other information about the project and the about page contains information about the development of the musical notation generator as well as how it works. There is also a user-guide for anyone wishing to use the application to follow to generate their own sheet music using it. It contains a short example recording of a piano as well as the expected results. A mock-up for the design could look like this:

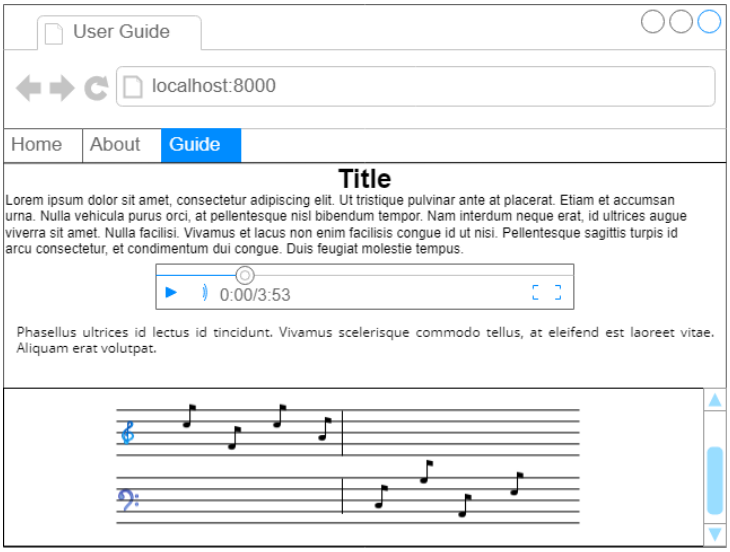


Figure 2.4 A mockup design for the user-guide

Audio Pre-processing

After the user uploads a file to the server, the system must then separate that recording into many smaller samples for each note. This is how the system is able to predict the pitch of each note individually. To separate each, note out, we must know how many data points are in each note. There are a few steps which can be taken to calculate this value. Firstly, the system must find the sample rate of the recording. This is how many data points are recorded every second in the recording. This value is a property of the metadata associated with the audio file when it is uploaded. Next, we must take the tempo of the piece of music to convert between real world time in seconds and the number of musical measure or beats. This can be used to calculate the number of samples taken for a single beat by using:

$$Samples\ Per\ Beat = \frac{60 \cdot Sample\ Rate}{Tempo\ (BPM)}$$

However, this will only allow for the smallest note division to be a crotchet, (or a quarter-note) and is suitable for some instances, but often a musician may need more precision. If a piece contains a quaver (eighth note) then the number of samples in each note will need to be halved. This information could be gathered from the user if they need this extra precision. This would not be automatic as the time taken to generate the score would likely be doubled which could reduce efficiency and accuracy drastically.

After the recording has been segmented down into the individual notes the backend will generate a spectrogram for each note. This is the exact same as when training the model so that this new data is of the same form to the training data. After these steps the recording has now been converted into a series of images ready for the model to make predictions on.

Inference

For systems like this one which utilise deep learning, a process must be developed to feed data quickly and efficiently into a model and then handle the predictions from the model in a meaningful way. This process is known as inference, and for it to work the program needs to send each spectrogram to the model and read the results while preserving their order. As discussed earlier, the output from the neural network is 128 different nodes, each of the nodes will output an activation value between 0 and 1. The node with the highest value will be the overall prediction from the network. However, this is only useful in single note inference.

Interpretation of Results

After a prediction is made using the deep learning model the results will be stored as a NumPy array of floating-point numbers with 128 indexes. Each of these values represents the model's confidence that the input sample is the corresponding note. Therefore, the majority of these values must be filtered out so that only notes that the model is confident in are kept.

For this project there are two types of filtering to consider. In a single note mode, the value with the highest confidence can be used to generate the sheet notation. The formula for this would be:

$$Note = index(\max(x))$$

$$Where:$$

$$x = \{n \in \mathbb{R} \mid 0 < n < 1\}$$

The output from this can then be converted into a note at that pitch for one note length. However, this is not as useful as a polyphonic solution which would allow this project to generate chords as well as single note melodies found in many popular songs. The best solution for this is to perform a filter on each of the results so that values with a high enough confidence are kept and the rest are ignored.

$$Notes = \{index(x) \mid x \geq t\}$$

$$Where:$$

$$x = \{n \in \mathbb{R} \mid 0 < n < 1\}$$

$$0.01 \leq t \leq 0.99$$

The final consideration is what the system will do when it encounters notes longer than a single prediction sample. If the raw output from the model was used, every note in the final sheet music would have the smallest note value which may sound very different from the original. Therefore, the system will need to detect when the same note is played consecutively and combine them together. Typically, in music these notes do not continue between bars, so the system should perform this analysis on a bar by bar basis.

Midi Generation

The penultimate stage before the results are delivered to the user are to convert the series of individual notes returned from the inference functions into musical data. Since 1981 there has been a standard developed for creating and storing music digitally the musical instrument digital interface. (Smith & Wood, 1981) In short, the midi file format stores information on what pitch is bring played and when it should be played based on a number of clock cycles. This is very similar to how raw recorded audio data is stored as discussed earlier. For this project, the system needs to create midi messages for when each note must begin and end. These are known as message on and message offs. For each note the deep learning model detects, a midi message on will then be sent for that prediction and then a message off will be created at the next note.

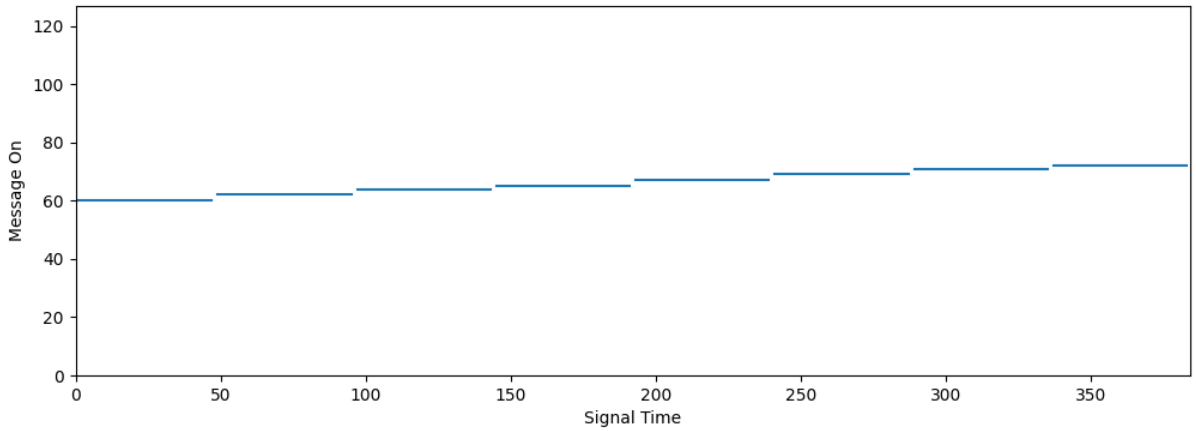


Figure 2.5 A visualisation of a series of midi signals

This resulting file is perfectly useable for opening into musical editors and generating sheet music there. A more experienced could use this to modify this midi file in a specialised software package such as a digital audio workstation. Although this is not the final deliverable to the user as a midi file is not readable by itself and requires another step before it can be performed directly.

Score Generation

The final action for the musical notation app to perform is to convert the midi file into a standardised traditional format. This can be done using an external application such a lilypond which is a program for musical engraving and visual music generation. Lilypond can be used in conjunction with a python package such as music21. (Lilypond, 1996) This toolkit could be used to call a visualiser framework and convert the midi file generated earlier to a pdf. Finally, this pdf file can be embedded back into the web interface. Once the pdf has been sent back to the client side the user should be able to download this file before the system is reset ready for another sample.

3.0 Design

The purpose of this section is to discuss the ideas behind the design of the final solution. The design process for a deep learning solution such as this involves many different steps which build on top of one another. Each individual component must be designed separately as single units before being integrated into their larger ecosystems. These larger solutions can then be used in the final deployment of the application.

3.1 Architectural Design

The first design consideration is the flow of the program and how each function will interact with each other. Achieving this are two pipelines which map how to data is modified through each process. The first of the process flows is for the model development as specified in the previous section.

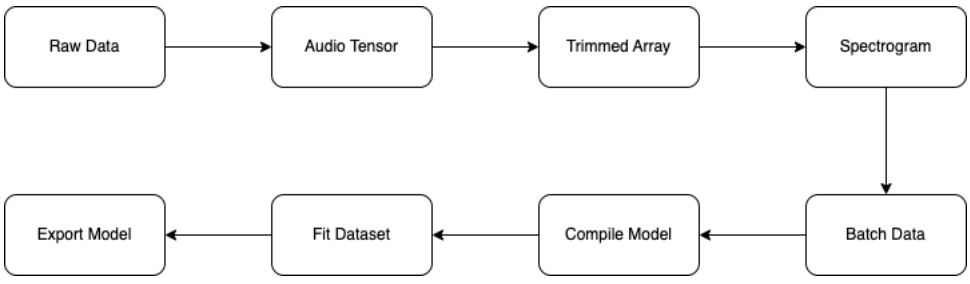


Figure 3.1 The process flow for model training

Here we can see the individual steps in generating a deep learning model. The point of consideration then when linking these units together is ensuring the output from one section is compatible with the input of the next.

From Raw Data to a Batch of Spectrograms

As mentioned in the specification section, the raw data at the start of this process is a directory with hundreds of thousands of uncompressed audio files. Therefore, the system must convert these into a format which is suitable for sequencing when fitting against the model. The best method for this is using the io library from TensorFlow, this can be used to create a tensor data structure from the file. A tensor is a way of arranging data similar to a matrix, though it has some advantages when computing with them as they can be arranged more efficiently through parallelisation where more data is read simultaneously. These tensors are also ready to be used in convolutions which will be explained in the model design. These audio tensors must also hold other information about each sample trough annotating them. These tensors can then be pre-processed, and their spectrograms can be generated before being batched. These batches are TensorFlow datasets which are very efficient ways of storing large sets of data and are also directly compatible with the input layer of the model.

Fitting data to the model

After the dataset is fully prepared and arranged in sequence in a dataset object, it can be loaded in by the program and batched. Batching is a more complicated process involving pre-fetching and memory placement, it will be discussed further in the implementation chapter. The model can now be defined and compiled using Keras before training the model using a simple prebuilt training loop. During training the progress of the model can be reviewed by making call-backs to the Tensorboard library. Tensorboard is a tool from TensorFlow to visualise and better understand the metrics recorded by the model over time. Once the model has reached a high enough accuracy as required by the specification, it can be exported and saved as a `saved model` format.

The other architectural design is the inference system that is used to make predictions. The system is based on a python Flask server hosting a website to upload audio to be processed and make calls to the backend to perform tasks.

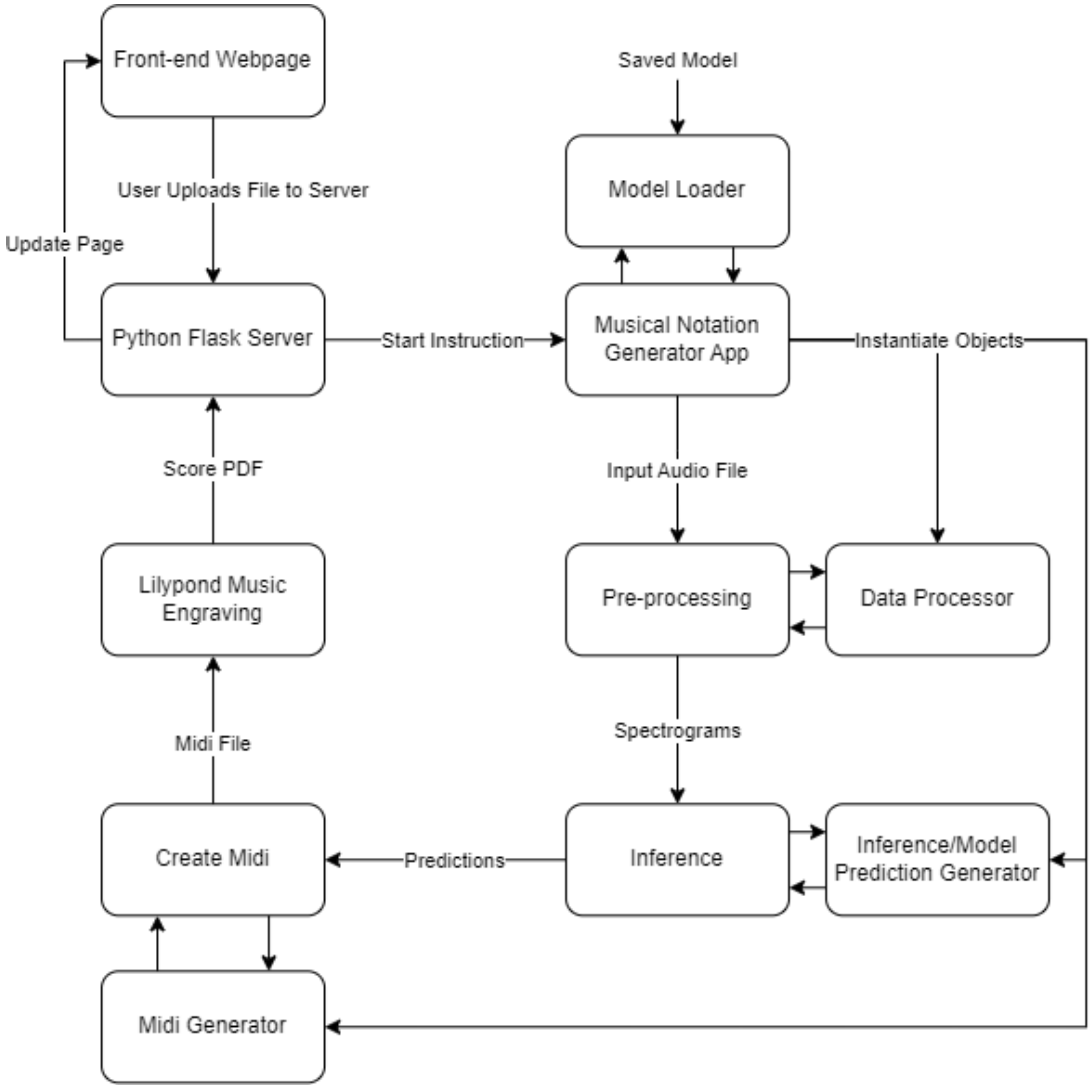


Figure 3.2 The process flow for the inference system

Here we can see the basic flow of the system may seem quite complicated. However, it can be simplified by following the data pipeline, which is split into distinct sections.

Website Interface

For this project, the way that the user can interact with the application through a web-interface. The requirement for this application is only a simple website with three pages and the ability to upload a digital audio file to the server through a form. This section will discuss how the frontend webpages communicate with the server. The webserver is written in Flask which is a lightweight application framework for python. A reason for using flask in the design for this application is how the webpages load dynamically, this python script will therefore only need to point to a directory with the html files for each page. Since the user is served dynamically, if any of the pages need to update, their markup files can simply be

modified. This is very simple but perhaps not the most efficient way, however a more complex solution is beyond the scope of this project.

The website consists of three pages, the home pages where the application can be accessed. An about page with information about the development of the project as well as its creator and a link to this document. The final page is a user-guide with an example recording and expected results. Of these three pages only the home page is dynamic and will change during runtime. The homepage consists of an html form to allow the user to upload a file as well as other information about the sample. The python script will then handle this file in the server-side filesystem. The system will also append additional metadata to this file based on the other user-given information. Once this file is uploaded to the server the model predictions are ready to be made.

Application Call

The webserver is able to run the application automatically, this is similar to emulating a user clicking on an executable file. This application can then return musical notation as a pdf automatically, as the file it needs to process will already be in a fixed location in a suitable format as it has been handled by the frontend. The system will load this file from the user and then continue to operate as any normal object orientated program would. A visualisation of this connection would be as follows:

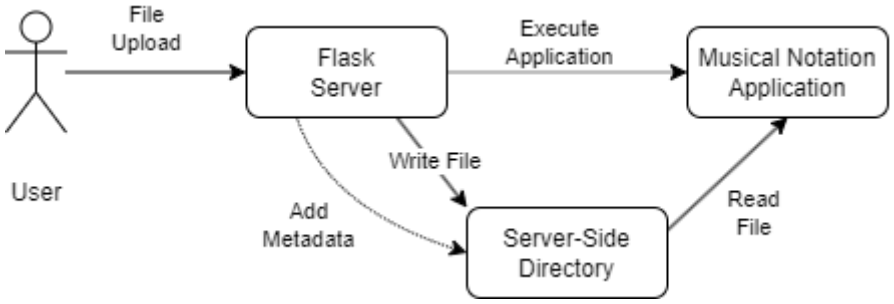


Figure 3.3 A use case diagram demonstrating the GUI.

Object Instantiation

The main function of the application can be broken down into three major steps as seen in the process flow diagram. Data pre-processing, inference and converting predictions into a score. Each of these steps can actually be thought of as a simple process which is repeated many times, usually a spectrogram is generated, predictions are made on it and it is converted into midi one note at a time. Therefore, it would only be logical to employ each of these tasks to objects. The concept of object orientated programming is to increase efficiency

when compared to linear or procedural programming where instructions are executed in sequence. In OOP an object is created using a constructor which then has attributes about it to define wider parameters, each of these attributes can then be used in a task an object performs. In procedural programming functions are used to define individual tasks to return a value. An object can perform these tasks more efficiently on your behalf through its methods.

The first object instantiated by the application is the data pre-processor. This object must be able to load the audio file and perform the necessary transformations before generating a number of spectrograms to feed into the deep learning model. The next object is the inferer, which is responsible for managing the model and how it interacts with the rest of the data in the pipeline. This is where the predictions are made using the model. The final object is the midi generator. This object takes the results from the model as an input and interprets them into note values based on a confidence value determined by the developer. It must then arrange these notes into a sequence, known as a bar in musical terminology and then generate midi signals from these predictions. After this midi is generated for every bar, the score can be engraved and sent back to the user. A preliminary UML diagram for these objects follows:

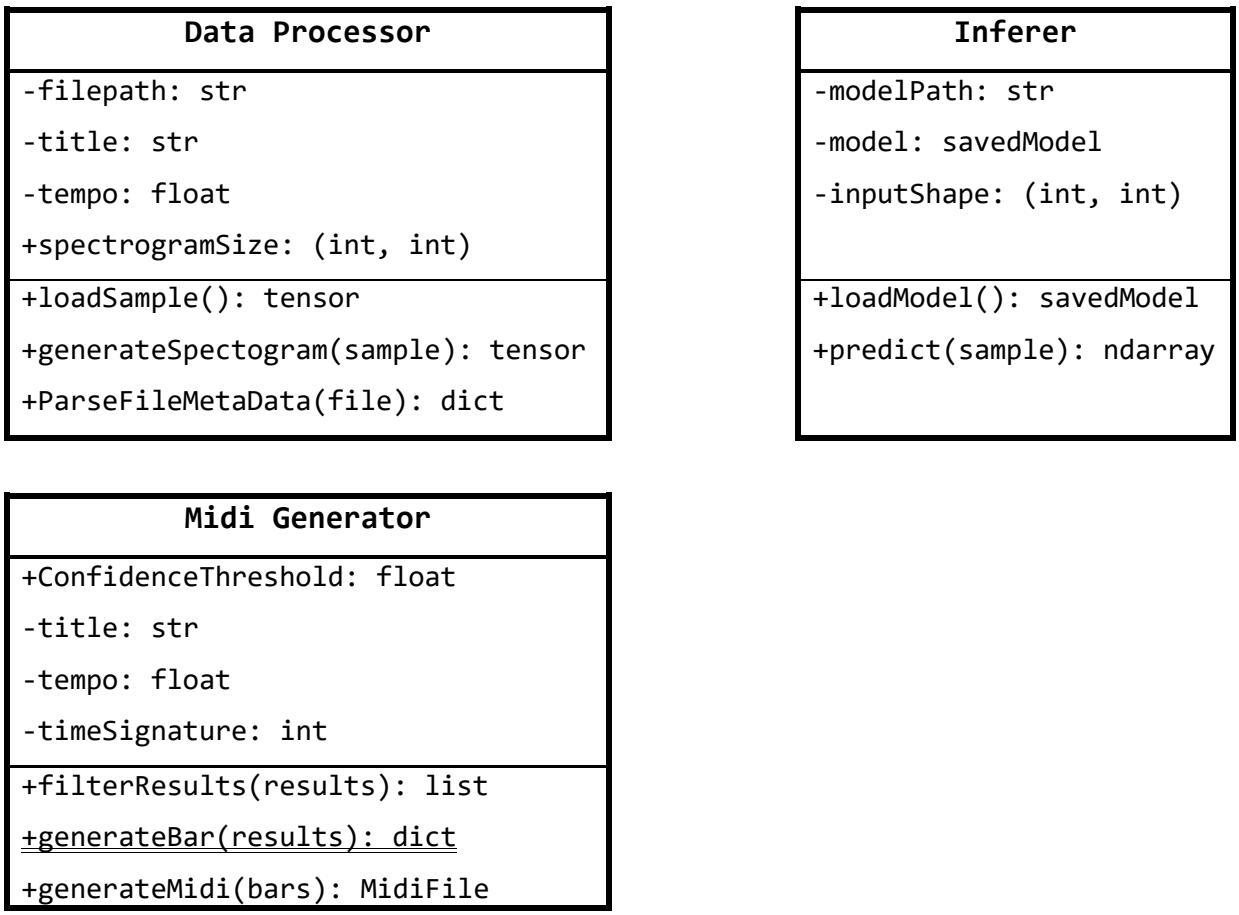


Figure 3.4 A UML class diagram for the runtime object of the application

Inference

After the program has started and all of the necessary setup has been completed, the results can be generated. A more detailed view of how the output is generated can be seen below:

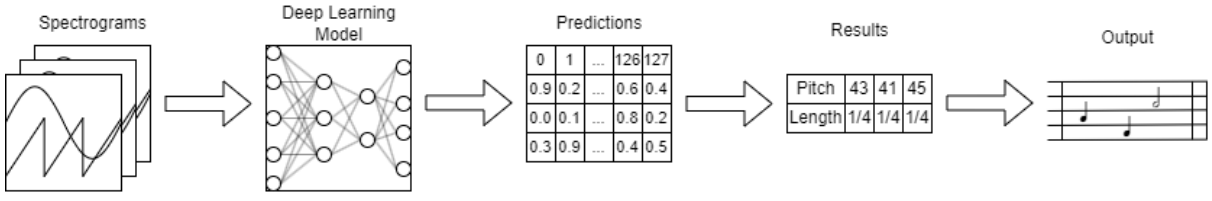


Figure 3.5 A pipeline diagram explaining how the application performs inference.

As we can see the data follows a linear path when all of the samples are batched together. Therefore, the main concern when connecting each of these sections is that the datatype is preserved to ensure compatibility. For this NumPy arrays are used to handle each individual note as they are very efficient due to being written in C. (Harris, et al., 2020) The other advantage of using these arrays is that they are mappable, this means the same function can

be applied to every note in the array individually but also retaining the efficiency from earlier. This is in turn used to convert each note into a midi signal by mapping the results array. The final output is generated as a pdf file before being sent back to the homepage, the application is then finished and stops executing.

3.2 User Interface Design

As discussed in the previous section the user will interact with three separate webpages. Each of these pages must be designed to fit their specifications and perform their functions. Each of these pages are designed in HTML so that they are compatible with any web browser. Each page also utilises a CSS reference file for styling to change the appearance of the site. The CSS styling is also used to make each page more accessible by improving the layout and making the text more readable.

Home Page

For the home page to be an effective interface for the inference application it is required to gather information from the user and send it to the server. For the system to make predictions on a recording it needs certain information. To package this information an html form can be used as this has been the standard method for over 25 years. This form must contain a file input box for the user to upload a file to. The html input tag with the file attribute can be used to achieve this.

```
<input type="file" id="myfile" name="myfile">
```

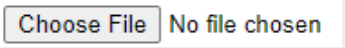


Figure 3.6 An example piece of html code showing a file input

This is the main feature of the page and therefore should be large and at the forefront of the user's attention when first loading the page. The other variables required by the system are the time signature and tempo of the recording, as well as how precise its analysis should be. These options can all be incorporated into the existing form which is used by the file upload. The time signature and level of precision are both single values which can be chose from a list. The best way to represent these then would be a series of radio checkboxes. This are easily understood by the user and can scale very nicely if more options are required. Below is a design for these inputs in html.

```
<p>Time Signature</p>
<input type="radio"
      name="time_signature"
     value="44"
     checked="checked"
/>
<label for="44">4/4</label>
<input type="radio"
      name="time_signature"
      value="34"
/>
<label for="34">3/4</label>
<input type="radio"
      name="time_signature"
      value="68"
/>
<label for="68">6/8</label>

<p>Note Division</p>
<input type="radio"
      name="note_division"
      value="0"
      checked="checked"
/>
<label for="Crotchet">♩</label>
<input type="radio"
      name="note_division"
      value="1" />
<label for="Quaver">♪</label>
<input type="radio"
      name="note_division"
      value="2"/>
<label for="Semi-Quaver">♪</label>
```

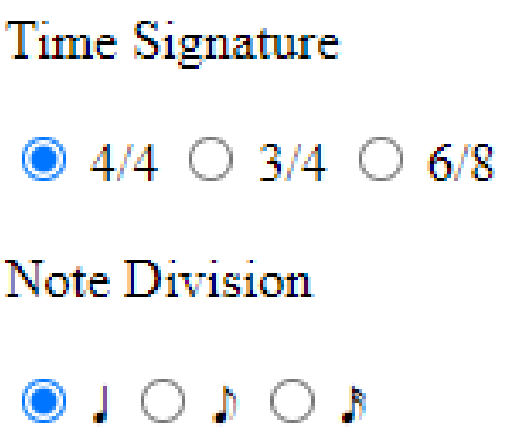


Figure 3.7 An example html radio buttons for the user to make selections from

The final piece of information required form the user is the tempo. A simple text input would be a viable option for this case as it is very simple to implement and allows the user to enter any keyboard input. However, this does present an issue of data validity when this information is passed into the application. The alternative to this is using a number spinner box which is a standard html element but has the added benefit of only allow the user to enter valid numbers into the box. This prevents errors during the inference process as the tempo is required to be either an integer or floating-point number. This is for when the tempo is used to calculate the size of the spectrograms.

```
<p>Tempo in bpm:</p>
<input
      type="number"
      value="120"
/>
```

Tempo in bpm:

120

Figure 3.8 An example of an html number spinner box

The final design feature of this home page is how the results are displayed to the user. The final output of the application is the musical score. There are various ways that this data could be represented and displayed to the user. However, the greatest way to store this file is a pdf as this filetype is compatible with more devices and is very easily transferable to many devices such as mobile phones. When delivering this file to the user there are two main approaches that are usually taken. Usually, to deliver a file of any generic type to the user using http requests. This is not very convenient as it requires the user to the open that file, and sometimes a viewer is not available for that file. Instead for this project the resulting musical notation file can be embedded directly into the webpage as a link. When combined with the dynamic loading of the Flask server, when this pdf file is changed in the filesystem the webpage will display the results automatically. This allows the user to preview the results directly in the webpage. When all of these elements are brought together in a final html file, the resulting webpage appears with all of its functionality. Although it is not very accessible. That is why a CSS styling guide is necessary.



Figure 3.9 The appearance of the final homepage without any styling

About Page

As a part of the website, the user should be able to find out more information about this project. This page simply contains one or two paragraphs explaining the purpose of the project and its creator as well as a brief overview of how it works. In the about section there is also a link to the git repository which hosts the source code for this project. Allowing anyone to download and run their own instance of this application or to train their own model using the instructions in the code. This page shares a stylesheet with the rest of the website to create consistency between the pages.

User-Guide

The final page to be designed for the user interface is the user-guide. The purpose of this section is to explain to any users who wish to use the application how they can generate their own results. As well as providing an example case for users to test the system without their own recordings. This page should also have extra information about the project and how a user can host the project themselves or contribute to its development.

Final Designs

The final designs for these three pages can be seen below. Each of these were developed using HTML and incorporate a style guide written in CSS with a basic shared colour scheme and text style throughout.

Home Page

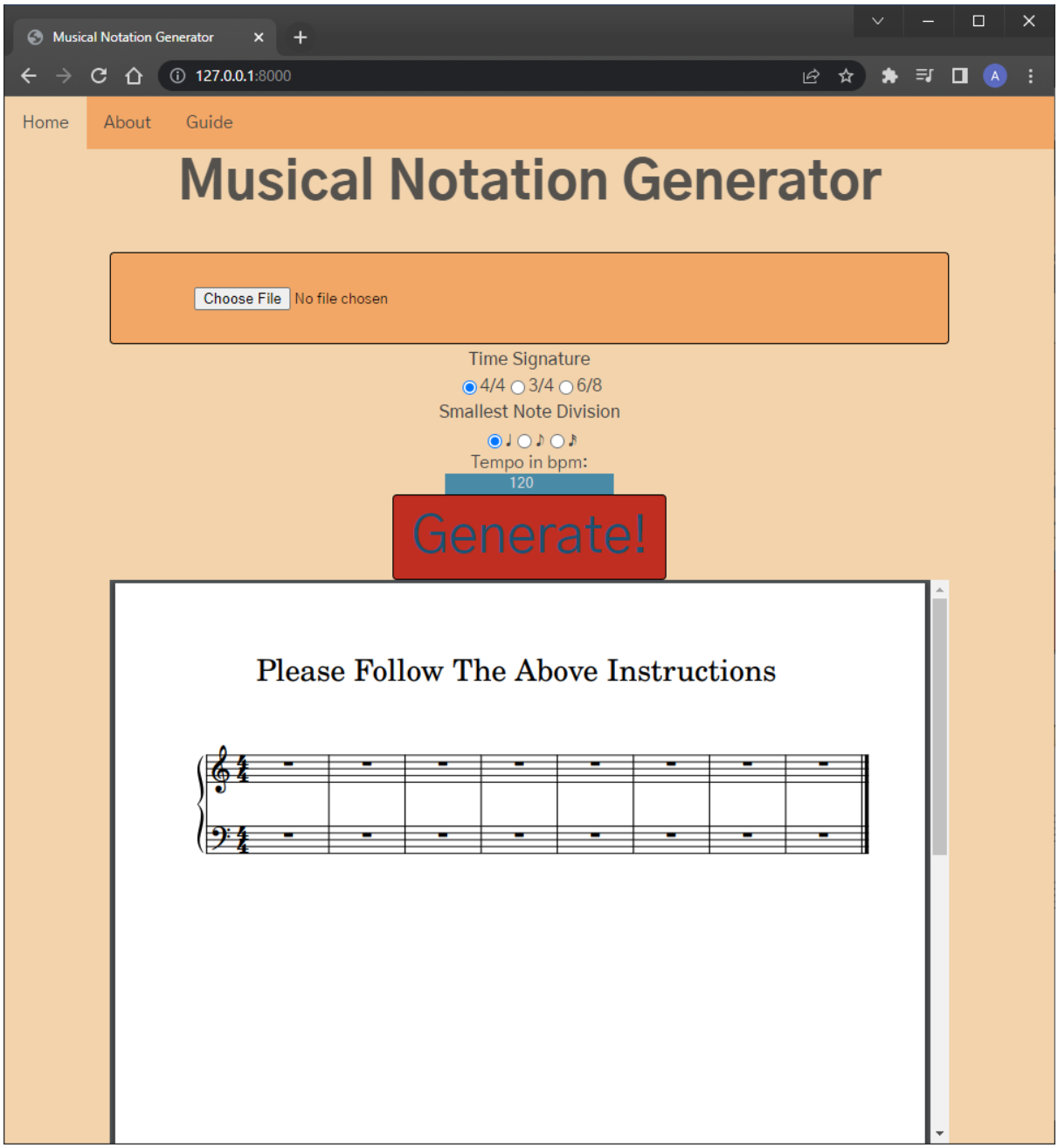


Figure 3.10 The final homepage with styling

At the top we can see the navigation bar which has been transposed horizontally and is consistent across the entire site, followed by the form with a series of inputs for the user before the submit button at the button to upload their responses to the server. At the bottom of the page, we can see a default score. This will be present at the start of each new instance of the server as the Flask server which hosts this site also performs a few background tasks. The affect of the default score appearing is due to the server managing its own filesystem by removing the input and output files regularly. This also provides benefit to the user as they do not need to worry about their information being stored long term.

About Page

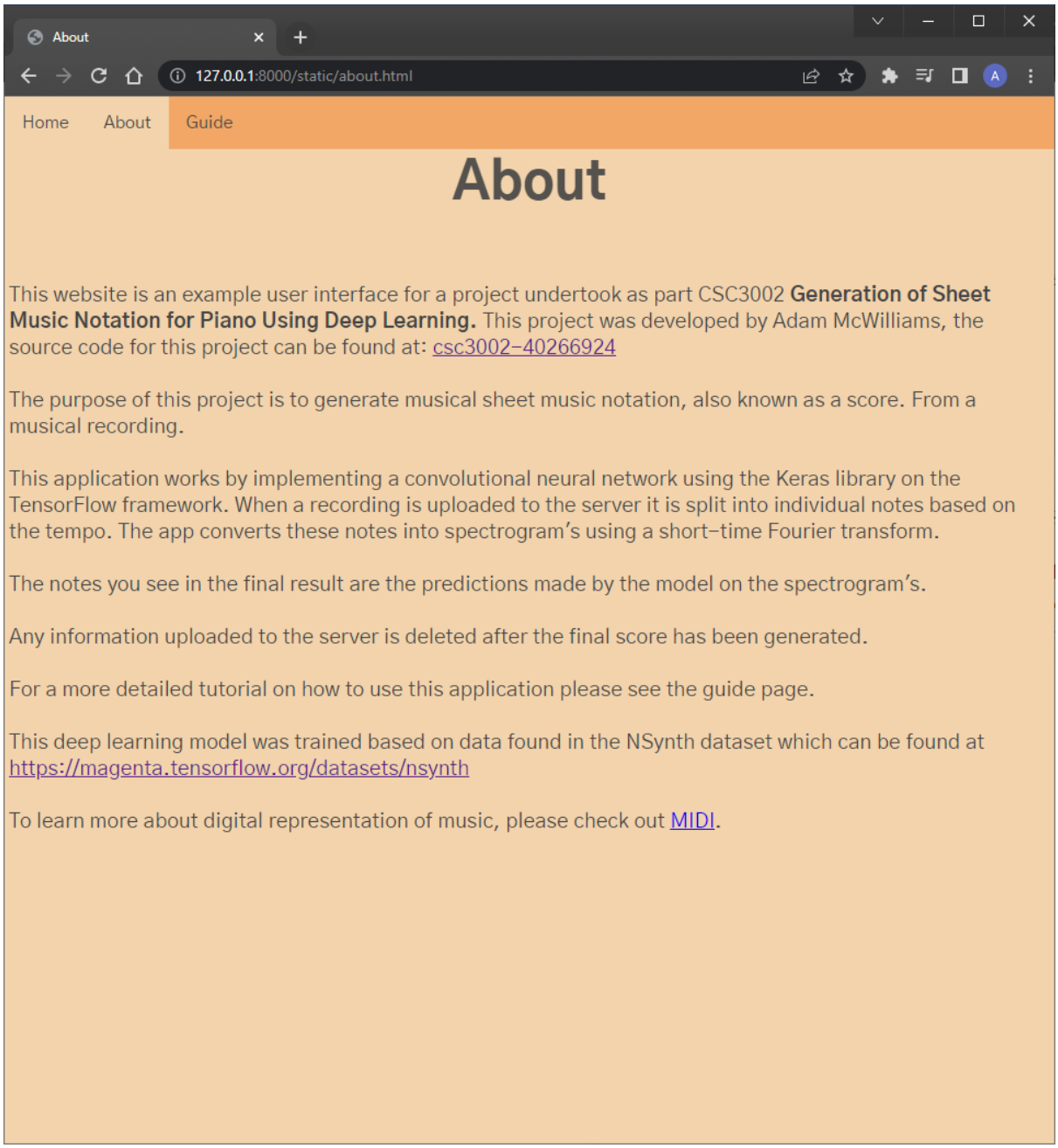


Figure 3.11 The final about section page

Here we can see the about page with a full description of the project and a brief overview of its development. There are also links to external resources if the user would like more information about the components that build the application.

User-Guide

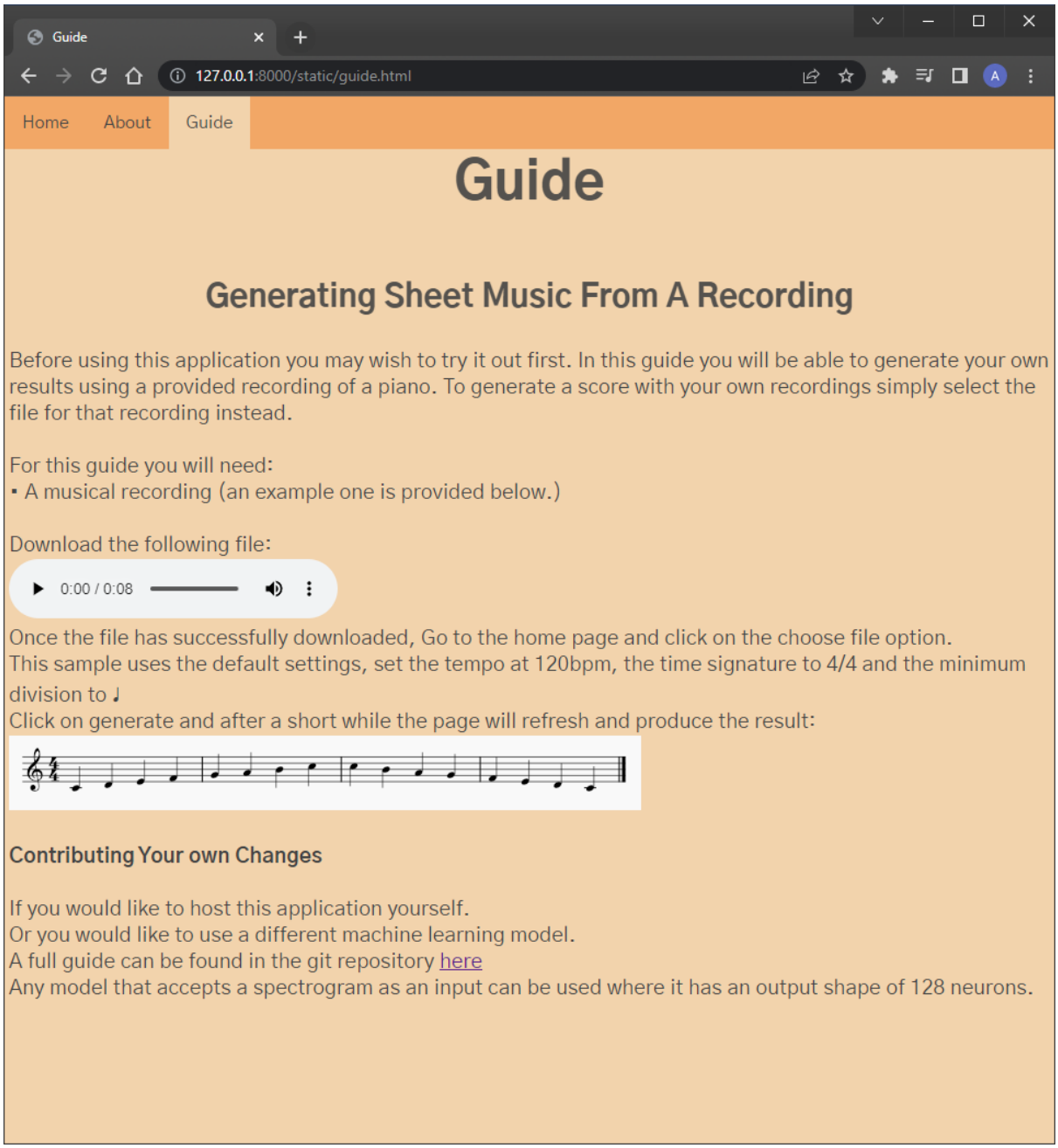


Figure 3.12 The final user guide

The final page shows a simple set of instructions for the user to perform their own test of the system. The main point to note on this page is the audio file and image of the expected outputs. These are able to be loaded dynamically by changing the files within the git repository. This allows for more complex examples to be used overtime as the model development improves.

3.3 Component Design

In a previous section the architectural design laid out the overall flow of the system during its runtime. Now that the user interface and front end have been designed, the process for understanding the musical files and converting them into sheet music is next.

Model Loading

The primary functions of the model loader are:

1. Check the validity of the file path where the model is stored.
2. Check if the model can be loaded properly.
3. Load the model into memory.
4. Determine the shape of the input layer of the model.

Now that these requirements have been established it is clear how this unit will need to be designed, and each of these points can be tested precisely. The first point can be achieved using the python OS module. When the model loader is given a file path, the OS module can query the filesystem to see if that path exists and return a Boolean. If the path isn't found a message should be return to prevent errors. The rest of the requirements can be fulfilled using the Keras library and querying the model itself. When loading the model, if the file path is not a valid model format, either a Keras saved_model directory or a .h5 file, the loader will raise an OS error. This is an essential piece of information as it can tell the developer or system administrator of a model has become corrupted.

The other benefit of this is seen directly through the efficiency of Keras, since if an OS error is not raised Keras has automatically loaded it into memory. At this point the model is ready to make predictions. However, with one of the system requirements being that the model and inference applications are not dependant on one another. To allow for this, the system must be able to dynamically change the size of the spectrograms it generates to fit whatever model is given. This can be done by querying the input of the model by using the shape attribute.

Pre-processing

The pre-processing section of the code will be the first instance of the objects that were mentioned earlier in this chapter. From the UML diagram and process flows it is clear that this object must be able to:

1. Read and parse the audio file that is uploaded by the user.
2. Load the other attributes uploaded by the user and validate them.

3. Divide the audio recording into smaller sections, each of these should be exactly one beat long.
4. Generate one spectrogram for every note in the audio file.
5. Batch these spectrograms together ready to be fed into the model.

The file can be read using a python library dedicated to handling audio files such as Wave or SciPy. Before it can be processed into spectrograms though it must be converted into a single signal. A popular recording technique is stereo recording, which separates the audio signal into two channels for playback through left and right speakers. Whilst this is great for listening to music, it can be challenging for analysing the individual components of the music. To overcome this, the left and right channels can be combined and averaged to produce a more accurate mono signal for analysis.

To gather information about a recording file provided by the user, the system can manipulate the metadata of the file. In addition to the information contained within the file, digital files stored on computers have associated metadata that describes various attributes about the file. By uploading a file to the server through the user interface, the system can create metadata that contains the user's responses about the recording from the rest of the form. This metadata can be parsed later using a Python library such as Taglib. (Wheeler, 2004) By combining the mono signal with other information, such as the tempo, we can convert between note values and the number of samples in the recording.

After the mono signal is generated and the other information such as tempo and duration are extracted, the system can begin generating spectrograms to use as input to the model. The number of spectrograms generated is variable and depends on the number of musical notes in the recording. The process of generating the spectrograms must be identical to the method used during training of the model, to ensure consistency in the input data. Once the spectrograms are generated, the system must trim them to ensure compatibility with the input layer of the model. This trimming process involves making sure that all the spectrograms have the same dimensions. This is due to the fixed size of the input layer of any convolutional neural network.

Inference and Midi Generation

The most critical aspect of this application is the inference section, which serves as the core component of the entire system. It can also be the most challenging area of the code. One of the most significant concerns is preventing errors by managing the data correctly.

When making predictions using a deep learning model, even the slightest deviation in the data format can lead to numerous potential errors, making it crucial to ensure that the data is in the correct format.

This is why the previous design sections are so essential, as they ensure that the data is consistently prepared and formatted correctly, reducing the risk of errors when making predictions. The inferer object plays a vital role in this process, as it is responsible for taking the spectrograms generated by the previous components and feeding them into the model.

One of the inferer's primary responsibilities is to retain the order of the spectrograms and return a set of confidence values that correspond to the model's predictions. Therefore, the inferer must operate seamlessly and efficiently, ensuring that the output data is accurate and reliable.

The predictions which are returned by the model are stored as a matrix of confidence percentages. This can appear very confusing at first, so the following diagram attempts to simplify this data.

Confidence (%)		**Pitch Detected**						
		0	1	2	…	125	126	127
Beat	1	0.00127	0.00249	0.01469	…	0.05230	0.83824	0.00000
	2	0.01540	0.00000	0.00500	…	0.90000	0.00542	0.00000
	3	0.00543	0.00743	0.00001	…	0.00432	0.00132	0.00485
	4	0.00000	0.00000	0.00000	…	0.00000	1.00000	0.00000
	5	1.00000	0.00000	0.00000	…	0.00000	0.00000	0.00000
	6	0.30014	0.00432	0.05310	…	0.00023	0.00074	0.00052
	7	0.00000	0.00000	1.00000	…	0.00000	0.00000	0.00000
	8	0.15090	0.75390	0.09532	…	0.00002	0.00034	0.00674

Table 3.1 Potential predictions from the model, cells in green are above an example threshold of 70%

The next step is for the application to transform the matrix generated from the inference into a format that can effectively represent musical notes. In conventional deep learning solutions, it is often sufficient to select the neuron with the highest confidence value as the prediction. The index position of this value would then represent the final prediction.

However, this approach falls short in meeting the requirement for the system to be polyphonic. To address this, a threshold value can be introduced, allowing for multiple predictions per note if the model has a high degree of confidence in those notes.

At this stage of the process, we have successfully converted an audio file into an array of pitches, which satisfies the requirement of automatically detecting the pitch of notes. The next step is to determine the length of each note. To do this, it is important to understand time signatures and their relationship to a bar of music. Every piece of music has a tempo, which is the speed or pace of the music. This tempo is expressed as a number in beats per minute, and the length of each beat is determined by the time signature of the piece.

In music notation, the most common beat length is a quarter note or crotchet, which is represented by a 4 on the bottom of the time signature. The top number of the time signature tells the musician how many of these beats are in each bar. Typically, musicians play and think of music in one bar at a time, and this is also how the musical notation generator application understands the rhythm of a piece of music.

To make the prediction results more manageable and analyse them in a meaningful way, they must be grouped into individual bars based on the time signature of the music piece. For instance, if the time signature is 4/4 and the smallest note division is a quarter note, every 4 predictions from the model would make a single bar. Once the predictions are grouped into bars, the solution can use a compression technique called run length encoding to further analyse them.

Run length encoding is a method that replaces a series of identical values with the value and the number of times it occurs in the series. For example, if a sequence of predictions is [1, 1, 1, 2, 2, 3, 3, 3, 3, 3], it can be compressed to [(1, 3), (2, 2), (3, 5)]. This reduces the amount of data that needs to be stored and processed, making it more efficient. The resulting compressed sequence can then be used to represent the rhythm and notes of the music piece, which is essential for generating musical notation.

The penultimate step of the system is to convert the detected notes into a musical score in the form of a MIDI file. This can be achieved using a package such as midiutil, which allows for the creation of a single-track MIDI file. To create the MIDI file, the system needs to add each note to the track through its pitch, duration, and timing information relative to the other notes in the song. The resulting MIDI file can then be played back using a MIDI player or in this case passed to an engraved to visualise it as standard musical notation.

Score Engraving

After generating the MIDI file, the final step is to convert it into a human-readable music notation format that can be displayed on the web page. This can be achieved by using the music21 Python library to call the Lilypond application. Lilypond is an open-source music engraving program that generates high-quality sheet music from music notation data. The process involves writing the MIDI file to a temporary location on the server, and then calling Lilypond to that file path to generate a PDF file. This PDF can then be embedded in the home page and displayed to the user as is with no modification necessary.

3.4 Model Design

Now that the design process for the final deliverable application has been defined, it is crucial to specify the primary dependency of the system - a deep learning model. As outlined in the system requirements section, the model must satisfy several criteria:

1. The model must be completely independent of the system. The web application is designed to work with a saved model file.
2. The model must be able to accept spectrograms generated using the TensorFlow short-time Fourier transform method as input.
3. The model should be trainable on the NSynth dataset.
4. The model should be capable of predicting the pitch of a note and returning it in scientific MIDI notation form.
5. The model should demonstrate high accuracy across the training, testing, and validation datasets.

Data Pre-processing

To accomplish points 2 and 3, the dataset must first be prepared. The most effective approach to preparing the data is by creating a Jupyter notebook that facilitates data exploration and analysis. A Jupyter notebook is a programming environment that enables code to be executed in small snippets. This feature can be used to explore the dataset incrementally, with explanations and comments inserted between steps. This allows for a more comprehensive understanding of the data and aids in identifying patterns and insights.

The first step is to load the extracted dataset from the NSynth website. Then each of the data samples can be annotated and compiled into a single markup file. This file can then be used to verify that the dataset is appropriate for this task by ensuring the training dataset is diverse enough to prevent the overfitting problem.

After validating the dataset, it's time to generate spectrograms for each of the samples. Several parameters need to be fine-tuned through multiple testing iterations to obtain optimal results. Finally, the dataset can be put through the transformer.

To improve data management efficiency, TensorFlow datasets should be used. These store the spectrograms and their pitches in the tfrecords format. This is more efficient than storing hundreds of thousands of images in the filesystem by storing them as a single tensor. Additionally, using the tfrecords format is advantageous because it is compatible directly with the input of a Keras model.

Model Training

Designing and training a deep learning model can be challenging due to the random nature of neural networks. However, there are a series of steps that can be taken to define and train a model, although there is no guarantee that the resulting model will perform optimally. These steps include:

1. Defining the architecture of the model: This involves deciding how each layer in the model is connected to one another. This step requires a good understanding of the problem domain and the available data.
2. Defining each layer of the network: Once the architecture is defined, each layer of the network needs to be defined along with its parameters, such as the activation function and padding. This can be done using the Keras layers submodule.
3. Compiling the model: After defining the layers, the model needs to be compiled. This involves specifying an optimiser, loss function, and metrics to be used during training. The optimizer is used to tune hyper-parameters such as learning rate, which can have a significant impact on model performance.
4. Training the model: The next step is to train the model by fitting it against the training dataset. This involves running multiple epochs of training, during which the model adjusts its weights to minimize the loss function.
5. Evaluating the model: Once the model is trained, it needs to be evaluated using a validation dataset to assess its performance on data it hasn't seen before. The results can be used to fine-tune the model and improve its accuracy.
6. Saving and exporting the model: Finally, the trained model needs to be saved and exported to a suitable format for use in deployment. This typically involves saving the

model weights and architecture to disk as a file, which can then be loaded into memory during deployment.

By following these steps and fine-tuning the model architecture and hyper-parameters as necessary, it is possible to train a deep learning model that achieves a high enough accuracy the accurately predict the pitches of musical notes most of the time.

Model architecture

In Keras there are two main ways of defining a model's architecture, sequential or interconnected. A sequential model is the simplest type of connected neural network as the output of each layer acts as the input for the next layer. Interconnected models are much more complex as the weights between neurons or layers can be connected to any other neurons or layers. There is also no restriction on the direction of the network, so the model can also perform recursive operations by feeding the output of a layer into itself.

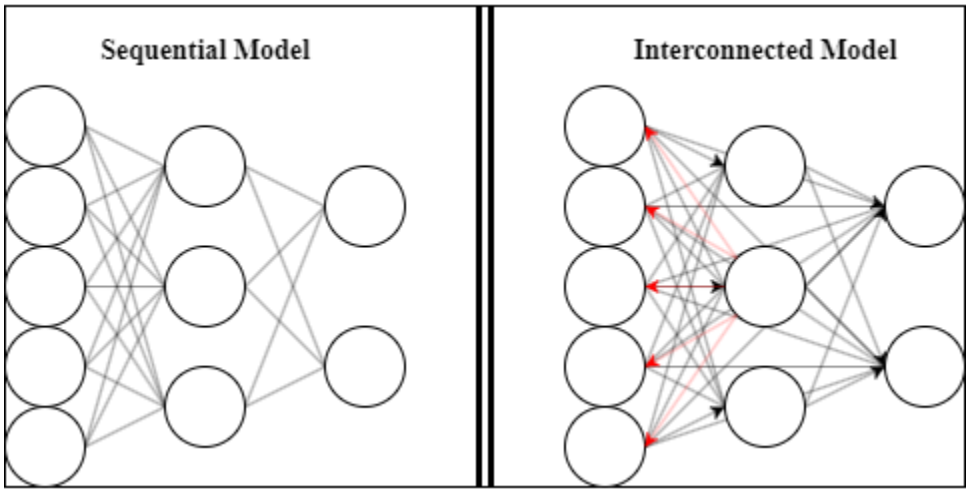


Figure 3.13 Two neural networks with different architectures. The network on the left has backpropagating weights in red.

A sequential model should be better for this use case as this project has a very well-defined input and output. These variables are also fixed after training and should not need to change. Therefore, the added complexity of using an interconnected design would not be beneficial as it would be much more computationally intense to train.

Layer design

A deep learning model at its core is a sequence of layers that transform input data as it passes through them. In Keras, there are many pre-defined layer types available in the library, such as convolutional, pooling, dense and dropout layers which we will explore. These pre-defined layers can be used in different combinations and configurations to create a wide range of neural network architectures.

To improve the design of a model, a developer can experiment with different layer configurations, such as rearranging the order of the layers or inserting new ones in between existing ones. They can also adjust the parameters of each layer to optimize the performance of the model. By experimenting with different layer configurations and hyperparameters, the developer can define a new deep learning model that performs better than previous iterations.

For this specific problem our input data is a series of images, this is why I have decided that a convolutional neural network should perform best. A convolution is a mathematical function which analyses a matrix and extracts features from it. (Xin & Wang, 2019) In the case of images convolutions are very good at detecting edges. This process can be done in our deep learning mode by adding a 2-dimensional convolutional layer into the network. This can then be followed by a pooling layer which in simple terms downsizes the results of the convolutional layer. These two layers can be repeated as many times as are needed until the output layer which can be a dense layer. A dense layer is a more traditional layer where each of the neurons are connected to every neuron in the previous layer.

A final type of layer could also be considered for use in this design known as a dropout layer. A dropout layer works by randomly activating and deactivating neurons in the network with each pass. These layers can be used to aggressively improve a model's performance against overfitting, however can increase the time taken to reach a high accuracy. These dropout layers should normally sit at the output of a pair of convolutional and pooling layers.

Different compiling optimisers

After the model has been defined there is a final step before it can be trained. The model must be compiled to define the training parameters. For this model the main concerns at this stage are the optimiser and loss function of the network. The optimiser is responsible for how the weights of the model are adjusted with each pass, while the loss function specifies the values by which the weights are adjusted. The most common optimisers for a Keras model are SGD and Adam, these are both gradient descent algorithms which attempt to generalise by extracting the most useful features of the images by changing weights which are 'more wrong' by larger amounts. (Kingma & Ba, 2017)

During training when data is fed into the network the output of the network is the prediction. The loss function is responsible for deciding how big the degree of error is between this prediction and the true value of the input. As this model is predicting between different

pitches of notes, we can call it a multi-class classification problem. For this type of model there are many different loss functions all of which should be considered as they work in different ways. The best way of determining the best one is to train various models and compare their performance.

Training, Validation and Saving

The final stage of the design process involves training and validating the model. This stage can be simplified by using Keras' predefined training loop. During training, a developer can monitor the model's progress by using Tensorboard, which displays accuracy and loss after each iteration. Using these metrics, a developer can decide if the model is performing well or not and if its training should continue.

After training, it's important to validate the model using new, unseen audio samples. During the pre-processing stage, 4,096 samples are set aside as a validation dataset. These samples are then fed into the model, and the resulting predictions are compared to their labels. The accuracy of the model is calculated based on these predictions, and if it well enough, the model can be exported and used for inference.

4.0 Implementation

The purpose of this chapter is to explore the themes and ideas used when implementing the designs of the previous section. This chapter will contain several examples of code used in the development of the musical notation generator application. A full listing of all code in this project is available in the repository and appendix.

4.1 Python for implementation

The backend application of this project in its entirety has been developed in Python. Python is an interpreted programming language which is an industry standard for machine learning development. This is because of the large amount of libraries available within Python for artificial intelligence development such as Sci-kit Learn, PyTorch and TensorFlow. These libraries can allow for advanced implementations of machine learning systems with very simple to develop and understand code.

Python is a widely used interpreted programming language in the field of machine learning due to its versatility and abundance of libraries. It has become an industry standard for developing machine learning applications. Python offers a wide range of libraries for artificial intelligence development, including Sci-kit Learn, PyTorch, and TensorFlow. These

libraries enable developers to implement complex machine learning systems with ease and reduce the complexity of the code through frameworks such as Keras. The flexibility of Python made it the ideal choice for developing a project like this.

Another advantage of Python which is used through the development of this project are its different environments. For developing the deep learning model, a Jupyter Notebook was used which allows the development process to be explained at each step.

4.2 Featured packages and libraries

Several libraries have been utilized throughout the development of this project, playing a crucial role in the backend application. Here are some key components that have been used:

1. Flask: a micro web-framework that is used to host the frontend user interface and to facilitate the connections between the frontend and backend of the system. (Miguel, 2018)
2. NumPy: a Python library that implements array data structures to increase performance and is used extensively throughout the project to arrange data and transport it between different sections.
3. TensorFlow: a machine learning library developed by Google, specialized in creating deep learning models from scratch. TensorFlow is the backbone of this project as it is used to develop, train, and make predictions with a deep learning model.
4. Keras: a framework that sits on top of TensorFlow and provides a level of abstraction in the model development. It allows for the use of pre-defined model structures and layers. (Chollet, 2015)
5. Midiutil: A Python package designed for handling digital musical information, which can be used to read and modify midi files systematically, making it a powerful tool. In this project, midiutil is used to generate musical information from a series of predictions. (Conway Wirt, 2016)
6. Lilypond: an application designed for musical engraving, used to improve the user experience. Lilypond converts the midi file generated by midiutil into a readable score that is easier to implement into the user interface.

4.3 Model Training in Jupyter Notebook

The following sections are taken from a Jupyter Notebook used to create the deep learning model.

Throughout this notebook there are several techniques used to improve efficiency when training the deep learning model. The first of these techniques can be seen here by using TensorFlow to find available devices to place data structures onto.

```
gpus = tf.config.list_logical_devices('GPU')

if gpus:
    strategy = tf.distribute.MirroredStrategy(
      gpus,
      cross_device_ops=tf.distribute.HierarchicalCopyAllReduce()
    )
else:
    strategy = tf.distribute.get_strategy()

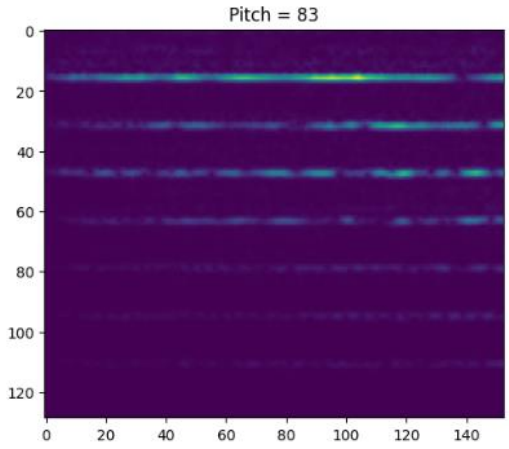
devices = [
      tf.config.list_physical_devices('CPU'),
      tf.config.list_physical_devices('GPU')
]

print(f'Found {len(devices[0])} CPU(s) and {len(devices[1])} GPU(s)')
```

```
Found 1 CPU(s) and 2 GPU(s)
```

To confirm the dataset contains usable data and to determine its size we will visualise one of the spectrogram's. The size of this spectrogram is used later to define the input shape of the deep learning model.

```
sample, label = trainData.take(1).as_numpy_iterator().next()
plt.figure()
plt.imshow(sample)
plt.title(f"Pitch = {label}")
plt.show()
```



This section is where the architecture of the deep learning model is defined. This code represents the 8th variation of the model. You can see the convolutional layers that make up the model. There is also an advanced technique used here, with strategy.scope():. This is using a distributed training strategy to spread the training evenly across all the GPUs evenly and uses the CPU for data management.

```
model_name = 'Model8'
tensorboard = TensorBoard(log_dir="logs/{}".format(model_name))
```

```
with strategy.scope():
    model = Sequential()
    model.add(Input(shape=sample.shape))
    model.add(Dropout(0.2))
    model.add(Conv2D(64, (3,3), activation='relu'))
    model.add(MaxPooling2D(pool_size=(2,2)))
    model.add(Conv2D(128, (3,3), activation='relu'))
    model.add(MaxPooling2D(pool_size=(2,2)))
    model.add(Conv2D(128, (3,3), activation='relu'))
    model.add(MaxPooling2D(pool_size=(2,2)))
    model.add(Conv2D(256, (3,3), activation='relu'))
    model.add(MaxPooling2D(pool_size=(2,2)))
    model.add(Flatten())
    model.add(Dropout(0.5))
    model.add(Dense(512, activation='relu'))
    model.add(Dropout(0.5))
    model.add(Dense(256, activation='relu'))
    model.add(Dropout(0.5))
    model.add(Dense(128, activation='linear'))
```

After defining the model it is now ready to be trained. To accomplish this we must first compile the model and define the training parameters.

```
with strategy.scope():
    optimizer = Adam(learning_rate=0.001)
    model.compile(
        loss='sparse_categorical_crossentropy',
        optimizer=optimizer,
        metrics=['accuracy']
    )
```

```
model.summary()
```

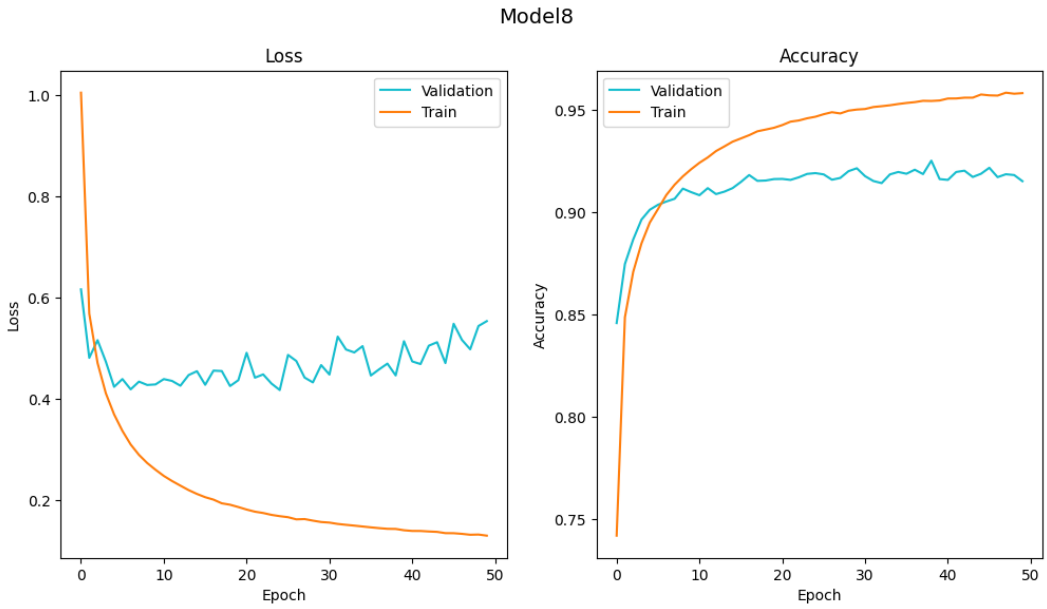
```
Model: "sequential"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
dropout (Dropout)            (None, 129, 153, 1)       0
conv2d (Conv2D)              (None, 127, 151, 64)      640
max_pooling2d (MaxPooling2D) (None, 63, 75, 64)        0
conv2d_1 (Conv2D)            (None, 61, 73, 128)       73856
```

```
max_pooling2d_1 (MaxPooling2D)    (None, 30, 36, 128)        0
conv2d_2 (Conv2D)                 (None, 28, 34, 128)        147584
max_pooling2d_2 (MaxPooling2D)    (None, 14, 17, 128)        0
conv2d_3 (Conv2D)                 (None, 12, 15, 256)        295168
max_pooling2d_3 (MaxPooling2D)    (None, 6, 7, 256)          0
flatten (Flatten)                 (None, 10752)              0
dropout_1 (Dropout)               (None, 10752)              0
dense (Dense)                     (None, 512)                5505536
dropout_2 (Dropout)               (None, 512)                0
dense_1 (Dense)                   (None, 256)                131328
dropout_3 (Dropout)               (None, 256)                0
dense_2 (Dense)                   (None, 128)                32896
=================================================================
Total params: 6,187,008
Trainable params: 6,187,008
Non-trainable params: 0
_________________________________________________________________
```

The model is now ready to be fit against the training dataset. During training the progress of the model can be viewed using tensorboard which can be accessed by running `tensorboard –logdir=logs/.` in a command prompt.

```
model.fit(
    trainData,
    epochs=100,
    batch_size=128,
    validation_data=validData,
    callbacks=[tensorboard],
    verbose=0
)
```

The following are charts produced from Tensorboard data for `Model8` over the first 50 epochs.



After the model is trained its final accuracy can be calculated against the test dataset.

```
predictions = model.predict(testData.batch(128), verbose=0)
_, labels = list(zip(*testData))
accuracyResults = []
for i, p in enumerate(predicitons):
      result = [labels[i]].numpy() == np.argmax(p)
      accuracyResults.append(result)

acc = round((accuracyResults.count(True)/len(accuracyResults))*100, 2)
print(f'Total tests: {len(accuracyResults)}')
print(f'Accuracy: {acc}%')
```

```
Total tests: 4096
Accuracy: 91.31%
```

```
model.save(model_name)
```

Here we can see that this convolutional neural network produced an accuracy of over 90% on unseen data and should perform well enough in most situations. The final output of this notebook is a Keras saved model folder. This model can be pointed at by the inference application.

4.4 Key features of Musical Notation Application

The other area of development in this project has been the inference solution. There are a few areas which I will explain here and the reasons for these approaches. However, the full annotated code can be found in the git repository.

Important functional elements

A function of this system is the connection between the frontend and backed components. This connection involves the sharing of files and executing scripts on the user's behalf. It is vital to the success of this project that the user is able to send extra information to the generator. This information can be collected by the front end using the HTML form. This information should not be sent directly to the application though. This is for system security as well as preserving the relationship between a recording and its metadata. To package these together, the following code manipulates the properties of the file and appends custom metadata to it which is preserved when transferred through different systems.

```
def tagMetadata(filepath: str, tags: dict) -> bool:
    """
    Appends metadata to the file uploaded to the server.
    :param filepath: The serverside filepath.
    :param tags: The metadata tags and values to append.
    :return: returns True upon completion.
    """
```

```
    with taglib.File(filepath, save_on_exit=True) as file:
        for tag in tags:
            file.tags[tag] = tags[tag]
    return True
```

Another essential area of the solution is interpreting the results from the deep learning model and converting them into meaningful values. The results of inference after they have been filtered are a sequence of individual pitches. The system must somehow find the relationships between each of these notes to generate information about their duration or when they occur. To achieve this, the following code has been implemented to group adjacent notes of the same pitch and calculate their duration. The result is a sequence of notes with their pitch and duration information, which can be used to generate a musical score.

```
def inferBar(bar: list) -> dict:
    """
    Generates midi signals for a single bar of music
    :param bar: A single bars worth of pitches
    :return: A dictionary of pitches and when they occur in the bar.
    """
    notes = set(chain.from_iterable(bar))
    result = defaultdict(list)
    for beat in bar:
        for note in notes:
            if note in beat:
                result[note].append(1)
            else:
                result[note].append(0)

    for key, value in result.items():
        result[key] = rle(value, len(bar))
    return result
```

Another advanced feature of this solution is its ability to handle variable input sizes. This means that spectrograms of any shape can be used as input to the model, and the system will automatically scale the dimensions of new spectrograms at the point of their creation. However, during testing, it was discovered that the system could not upscale smaller spectrograms, resulting in failed tests. To address this issue, the code was modified to ensure that the system could properly handle both upscaling and downscaling of spectrograms, resulting in a more robust and reliable solution. These changes will be discussed further in the testing section.

```
def getSpectrogram(self, sample: tf.constant) -> tf.constant:
    """
    Converts a tensor of audio signals into a spectrogram.
    :param sample: The data for a single note from the audio file
    :return: A spectrogram.
    """
    spectrogram = tf.signal.stft(sample, frame_length=256, frame_step=64)
    spectrogram = tf.abs(spectrogram[:self.targetFrameLength[1]])
    padding = [
        [0, self.targetFrameLength[1]-spectrogram.shape[0]],
        [0, self.targetFrameLength[0]-spectrogram.shape[1]]
    ]
    """
    Padding ensures that the spectrograms are compatible with the model.
    """
    spectrogram = tf.pad(spectrogram, padding, "CONSTANT")
    spectrogram = tf.transpose(spectrogram)
    return tf.expand_dims(spectrogram, axis=2)
```

5.0 Testing

To validate the development of this solution each section must be testing to ensure it both works and meets the specification.

5.1 Unit Testing

An advantage of using Python to develop this solution is that it gives us access to the unit testing library. Python unit tests are a method for automatically testing small pieces of code. There is a table containing each automated test as well as its result in the appendix. Full details can be found in the UnitTest directory in the code repository.

There are a few unit tests which failed, the following is a table with them and their solutions.

Test	Class	Description	Status
27	SpectrogramGenerator	Create an instance of the Spectrogram Generator object with missing attributes when using a non-audio file	Fail
28	SpectrogramGenerator	Create an instance of the Spectrogram Generator object with missing attributes when using a corrupt audio file	Fail
38	SpectrogramGenerator	Spectrogram shape should be same as model input	Fail

Resolutions

Tests 27 and 28 both failed for the same reasons. The taglib library was unable to read those files as they did not have the correct metadata attached to them. In the final system this issue would only arrive when the front-end does not perform as expected. However, this unit should still be fixed. The solution to the was to surround the metadata parser with a try catch

block and safely return an error message when a KeyError is raised for missing metadata, or when an OSError is raised when trying to read an invalid file type.

Test 38 failed because the size of the data used in the short-time Fourier transform was smaller than expected, resulting in a spectrogram shape that was smaller than the input shape of the model. This issue is significant because it prevents inference when the input and model are incompatible. To resolve this problem, padding with zeros should be added to the spectrogram to increase its size up to the correct shape for the model. The TensorFlow pad function is used to expand each axis of the spectrogram while preserving its datatype as a tensor.

All three of these tests have been retested after changes and now pass.

5.2 Manual Testing

There are a few features of the system which are not written in Python. This means that the user interface cannot be tested using an automated solution. For the website, manual tests were performed. In these tests environment conditions were laid out, with an expected outcome for the system. The full table can be found in the appendix, the tests that failed are:

Test	Page Tested	Description	Result
8	Home	Uploading a text file alerts the user to invalid format	Fail
9	Home	Uploading a corrupted audio file alerts the user that it is corrupted	Fail

In both tests the alert box failed to show. However, running the application in debug mode showed that the Flask flash method was sending a message to the homepage. The issue was in the rendering of this message. The following JavaScript was added to the home page HTML file to solve this. This code embeds python within the JavaScript allowing the webpage to read the python variables.

```
<div class="flash-message-box" style="min-height:100%">
  {% with messages = get_flashed_messages() %}
    {% if messages %}
      <script>
        var messages = {{messages | safe}};
        for (var i=0; i<messages.length; i++) {
          alert(messages[i]);
        }
      </script>
    {% endif %}
  {% endwith %}
</div>
```

5.3 Model Testing

Over the course of developing this project there were many different designs for the deep learning model. Each of these models have slight tweaks and differences to try and gain the highest accuracy. To test and compare each of these models the same 4,096 samples are used to make predictions on their pitches. The results of each model alongside a small description are below. Full results are available in the appendix.

Model	Description	Accuracy (%)
Model1	An initial CNN with 1 hidden layer	65.80
Model2	A similar model to 1, where training was placed on a single GPU and training time was decreased to 5 epochs	17.24
Model3	Same as model 2 but trained for 15 epochs	36.79
Model4	Changed Model 3 optimiser from SGD to Adam	80.37
Model6	Added Dropout layers in increase generalisation as well as many more convolutional layers	89.84
Model8	Increased is aggressiveness of the dropout layers	91.31

5.4 Integration Testing

The final way to test the system is to run it as a user would. This means simply running all components together and seeing the result. To do this an instance of the webserver is hosted and a sample audio file will be uploaded to it. Once the generate button is pressed, if all of the individual components work together a score will be shown to the user. The result of this test is shown in the appendix. The test is successful.

6.0 Evaluation

6.1 Final Evaluation

The final evaluation of the system can be measured by its success in three separate areas.

1. The deep learning model's ability to recognise the pitch of a given note from its spectrogram.
2. The ability for the musical notation generator application to infer notes and rhythms from an audio recording.
3. When integrating these two systems together they are able to generate sheet music from a given audio recording.

Model Evaluation

The testing results demonstrate that the model is highly effective at recognizing different pitches, achieving a prediction accuracy rate of over 90% on the testing dataset. This achievement fully satisfies the requirements of the project and serves as a strong foundation for further research in the field of musical audio processing.

Inference Evaluation

Through manual testing it has been shown that the inference application can efficiently interpret a musical recording and extract key features from it to infer rhythm from a sequence of notes. This system has also proved itself as very robust and is able to prevent errors in most cases and eventualities to improve the user experience.

Integration Evaluation

The integration of the two systems has proven successful with its ability to generate accurate musical scores from a recording. This system has provided the user with flexibility in the options that are presented to them. However, the system can be improved as it may struggle with audio files which hare recorded very differently from the training dataset. As touched on previously as a potential problem area, audio data is random by nature and there are far too many variables to accurate make predictions on every possibility. This is why for most cases of longer audio samples the system appears to perform very poorly. The system generates notes which all seem to be incorrect, however this is a result of fundamental frequencies in sound waves. This happens when the system is given audio that is not similar to the NSynth dataset, it will predict the fundamental frequency of that note. This means that a note will be predicted to be potentially several octaves lower than in reality. These results are not fully incorrect though as musical notes are cyclical and two melodies played in different octaves are the same to the human ear. This ability to detect notes in an octave separately from an octave is less than ideal but is still very impressive for a model which is relatively simple.

7.0 Conclusion

The ability for a deep learning neural network to detect and extract meaning from a musical recording is a very difficult task. However, it is possible to design and implement a solution which is efficient and does not require prior knowledge of artificial intelligence systems to use. Users are able to upload their own recordings in a simple to understand environment and generate useful results which can have an impact on their ability or experiences as a musician. There is still much more research that can be conducted in the area of music processing for machine learning. As techniques advance in the design of convolutional neural networks, we approach a more general solution to the problem of recognising music as a human could.

References

Git repository is available at: https://gitlab.eeecs.qub.ac.uk/40266924/csc3002-40266924

Chollet, F., 2015. *Keras.* [Online]
Available at: https://github.com/fchollet/keras

Conway Wirt, M., 2016. *MIDIUtil.* [Online]
Available at: https://github.com/MarkCWirt/MIDIUtil

Engel, J. et al., 2017. *Neural Audio Synthesis of Musical Notes.* s.l.:JMLR.org.

Harris, C. R., Millman, K. J. & van der Walt, S. J., 2020. Array programming with NumPy.. *Nature,* Volume 585, pp. 357-262.

Henschen, J., 2018. The Tragic Decline of Music Literacy (and Quality). *Intellectual Takeout,* Issue August 16th.

Kilmer, A. D. & Civil, M., 1986. Old Babylonian Musical Instructions Relating to Hymnody. *Journal of Cuneiform Studies,* 38(1), pp. 94-98.

Kingma, P. D. & Ba, J., 2017. Adam: A Method for Stochastic Optimization.

Lilypond, 1996. [Online]
Available at: https://lilypond.org/

McWilliams, A., 2023. *csc3002-40266924.* [Online]
Available at:

Miguel, G., 2018. Flask web development: developing web applications with python.

Nielsen, 2015. *Everyone Listens to Music, But How We Listen is Changing.* [Online]
Available at: https://www.nielsen.com/insights/2015/everyone-listens-to-music-but-how-we-listen-is-changing/

Smith, D. & Wood, C., 1981. The 'USI', or Universal Synthesizer Interface.

TensorFlow, 2015. *tensorflow.org.* [Online]
Available at: https://www.tensorflow.org

Wheeler, S., 2004. *TagLib.* [Online]
Available at: https://taglib.org/api/

Xin, M. & Wang, Y., 2019. Research on image classification model based on deep convolution neural network. *EURASIP Journal on Image and Video Processing.*

Yang, Z. R. & Yang, Z., 2014. Artificial Neural Networks. In: *Comprehensive Biomedical Physics.* s.l.:s.n., pp. 1-17.

Appendix

User Guide

CSC3002 - 40266924

Musical Notation Generator

Thank you for visiting the repository for my final year project.

Below are a few guides on how to use this project yourself.

Hosting The Web App

To host your own instance of the web application you must first install Python.
Once python is installed change directory into `resources`. Once there in a terminal window run.
`$ pip install -r requirements.txt` This will install all of the required packages for the application.
Next download and install both Lilypond and Musescore. Now in a Python instance run the following commands.

```
from music21 import *
us = environment.UserSettings()
us.create()
us['lilypondPath'] = $Lilypond_path
us['musescoreDirectPNGPath'] = $Musescore_Path
us['musicxmlPath'] = $Musescore_Path
```

Where $Lilypond_Path and $Musescore_Path are the file paths to Lilypond.exe and Musescore.exe

Now you should be ready to start the server.
In the `Application` folder run the `App.py file` to start the server.
The homepage should be visible at localhost:8000

Training Your Own Models

To host your own instance of the web application you must first install Python.
Once python is installed change directory into `resources`. Once there in a terminal window run.
`$ pip install -r requirements.txt`

This will install all of the required packages for the application. After the packages have installed change directory to the Training folder and run the command
 Jupyter notebook This will launch a Jupyter server.
Open the `Model Training.ipynb` notebook and follow the on-screen instructions to train a model.
To use your own model with the inference application, place the saved model folder in the `Application` directory and rename the file to `model`.

Thank you for reading!

Adam McWilliams April 2023

Testing Tables

Unit Tests

Initial Results

Test	Class	Test Name	Description	Status
1	Main	test_loadInferer_valid	Load a valid model into memeory	pass
2	Main	test_loadInferer_invalid	Load an invalid model, should return false	pass
3	Main	test_loadInferer_error	Load a model which does not exist should return false	pass
4	Main	test_inferer_shape	Test the models input shape	pass
5	Main	test_getInputFile	System should not run under these conditions	pass
6	Main	test_prepareData_type	Test that Spectrogram is a tensor	pass
7	Main	test_prepareData_spectrogramShape	Test that Spectrogram is compatible with model	pass
8	Main	test_prepareData_tags_type	Test that metadata has been parsed	pass
9	Main	test_prepareData_tags	Test metadata vakue	pass
10	Main	test_prepareData_valid	Prepare data should compile	pass
11	MidiGenerator	test_constructor_good	Object should be cerated	pass
12	MidiGenerator	test_constructor_bad	KeyError exception should be rasied as there is no metadata	pass
13	MidiGenerator	test_note_division	division should have been converted to a 1	pass
14	MidiGenerator	test_filterPredictions_valid	List of condfidence values should convert to valid pitches	pass
15	MidiGenerator	test_filterPredictions_empty	Should return list of empty pitches	pass
16	MidiGenerator	test_generateMidi_good	Midi file should be generated	pass
17	MidiGenerator	test_generateMidi_empty	Empty midi file should be generated	pass
18	MidiGenerator	test_inferBar_good	Dictionary of fixed lengths lists should return	pass
19	MidiGenerator	test_inferBar_empty	Empty dictionary should return	pass
20	MidiGenerator	test_inferBar_type	Inferbar should generate a dictionary	pass
21	MidiGenerator	test_rle_good	Run length encoding should condense list	pass
22	MidiGenerator	test_rle_variation	RLE should return compressed list with leading 0	pass
23	MidiGenerator	test_rle_gap	RLE should return list with values in 1st and last	pass

24	MidiGenerator	test_rle_empty	RLE should return empty list	pass
25	MidiGenerator	test_rle_type	RLE should always be of type list	pass
26	SpectrogramGenerator	test_constructor_good	Creates instance of object	pass
27	SpectrogramGenerator	test_constructor_bad	Create an instance of the object with missing attributes	Fail
28	SpectrogramGenerator	test_constructor_corrupt	Create an instance of the object with missing attributes	Fail
29	SpectrogramGenerator	test_filepath	Filepath should be same as input	pass
30	SpectrogramGenerator	test_filepath_exists	File should exist	pass
31	SpectrogramGenerator	test_title	Title should be parsed from tags	pass
32	SpectrogramGenerator	test_division	division should be parsed from tags	pass
33	SpectrogramGenerator	test_tempo	tempo should be parsed from tags	pass
34	SpectrogramGenerator	test_timeSignature	time signature should be parse from tags	pass
35	SpectrogramGenerator	test_targetFrameLength	single note window size should match model shape	pass
36	SpectrogramGenerator	test_getTags_good	Metadata values should be parsed correctly	pass
37	SpectrogramGenerator	test_getSpectrogram_shape	Spectrogram shape should be same as model input	pass
38	SpectrogramGenerator	test_getSpectrogram_smallerDivision	Spectrogram shape should be same as model input	Fail
39	SpectrogramGenerator	test_getSpectrogram_type	Spectrogram should be of type tensor	pass
40	Inferer	test_loadModel_Good	Keras saved_model should be loaded	pass
41	Inferer	test_loadModel_Bad	Function should return false	pass
42	Inferer	test_loadModel_NotFound	Function should return false	pass
43	Inferer	test_constructor_good	Object should be cerated	pass
44	Inferer	test_constructor_goodValid	ValidModel attribute should be true	pass
45	Inferer	test_constructor_bad	ValidModel attribute should be false	pass
46	Inferer	test_predict_good	Numpy multi-dimensional array should be created	pass
47	Inferer	test_predict_goodValid	Spectrogram should be accepted by model	pass
48	Inferer	test_predict_bad	Model should raise value error as spectrogram is incompatible	pass
49	Inferer	test_predict_badValid	Predict method flag should return false	pass

50	Utilities	test_checkFileType_Good	File type should be accepted	pass
51	Utilities	test_checkFileType_Bad	File type should be rejected	pass
52	Utilities	test_getFilePath_SafeFileName	Filename should be converted into path	pass
53	Utilities	test_getFilePath_NotSafeFileName	Invalid charecters should be removed from filename	pass
54	Utilities	test_setupFilesystem_static	Static directory should have Score.pdf file in it	pass
55	Utilities	test_setupFilesystem_variables	Variables directory should contain only Score.pdf	pass

Retesting results

Test	**Class**	**Test Name**	**Description**	**Status**
27	SpectrogramGenerator	test_constructor_bad	Create an instance of the object with missing attributes	Pass
28	SpectrogramGenerator	test_constructor_corrupt	Create an instance of the object with missing attributes	Pass
38	SpectrogramGenerator	test_getSpectrogram_smallerDivision	Spectrogram shape should be same as model input	Pass

Manual Testing

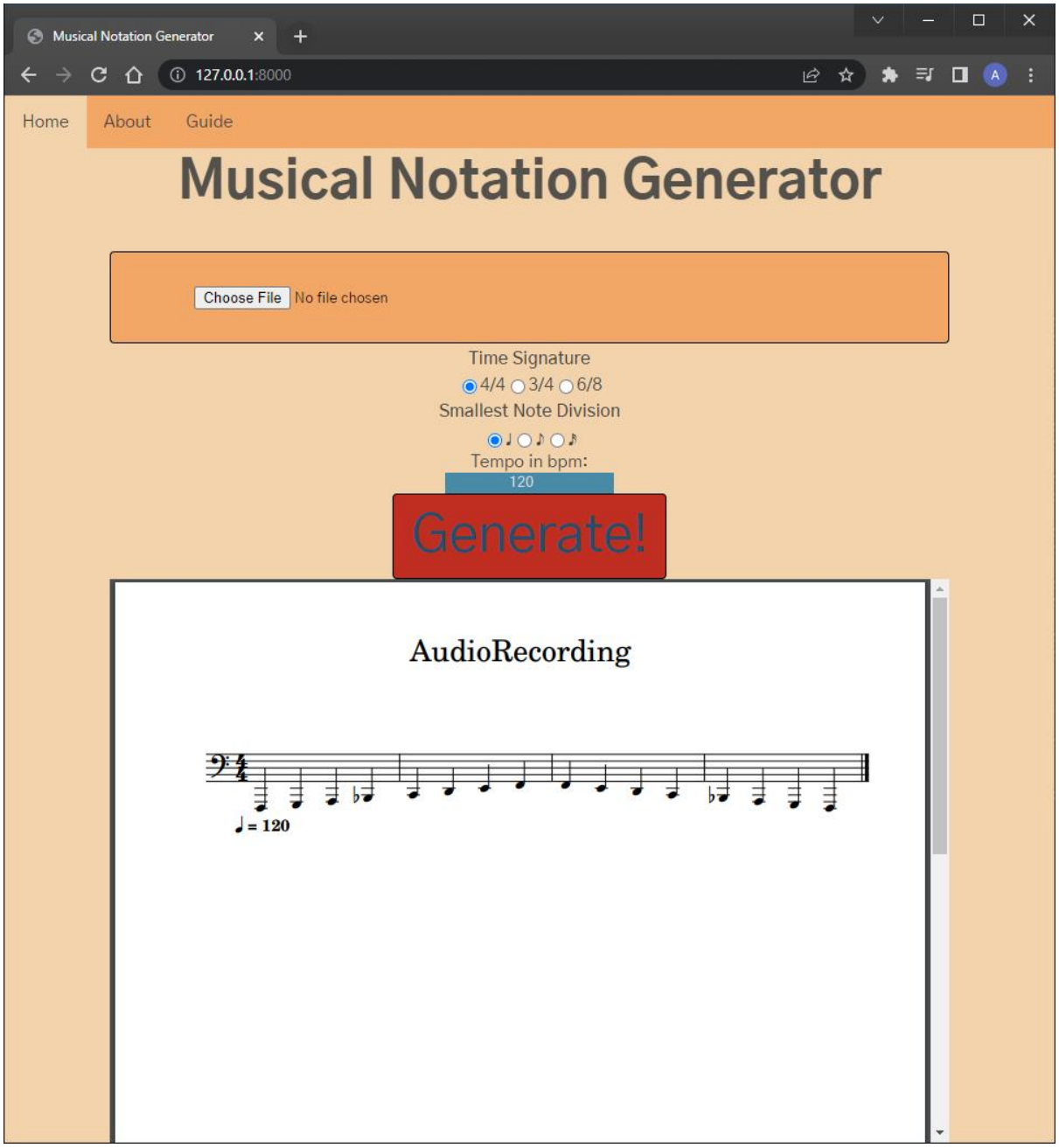
Initial results

Test	Page Tested	Description	Result
1	Home	About navigation button redirects to about page	Pass
2	Home	Guide navigation buton redirects to guide page	Pass
3	About	Home navigation button redirects to home page	Pass
4	About	Guide navigation buton redirects to guide page	Pass
5	Guide	Home navigation buton redirects to home page	Pass
6	Guide	About navigation button redirects to about page	Pass
7	Home	Pressing generate button without a file alerts the user	Pass
8	Home	Uploading a text file alerts the user to invalid format	Fail
9	Home	Uploading a corrupted audio file alerts the user that it is corrupted	Fail
10	Home	Uploading a valid file displays a generated score	Pass

Retesting results

Test	Page Tested	Description	Result
8	Home	Uploading a text file alerts the user to invalid format	Pass
9	Home	Uploading a corrupted audio file alerts the user that it is corrupted	Pass

Integration Test Results



Source Code

Application Code

App.py

In [ ]:

```
from flask import Flask, render_template, request, flash, redirect
import Utilities
import Main

app = Flask(__name__)

@app.route('/', methods=['GET', 'POST'])
def upload_file():
    """
    This function renders the home page and handles http requests.
    If any errors occur this function will display them
    in an alert box through flash
    :return: A http redirect to the home page
    """
    if request.method == 'POST':
        if 'AudioFile' not in request.files:
            flash('Form could not find file')
            return redirect(request.url)
        file = request.files['AudioFile']
        tags = request.form
        if not Utilities.checkFileType(file.filename):
            flash('File is invalid format')
            return redirect(request.url)
        filepath = Utilities.getFilePath(file)
        file.save(filepath)
        if not Utilities.tagMetadata(filepath, tags):
            flash('File could not be encoded properly')
            return redirect(request.url)
    """
    Main.main() is responsible for creating an
    instance of the backend application
    """
    result = Main.main()
    if result:
        flash(result)
    return render_template('index.html')

if __name__ == '__main__':
    """
    This is the entry point for the web server.
    A Flask app is configured to run on port 8000
    A secret key is setup to allow POST requests to the server.
    """
    Utilities.setupFilesystem()
    app.secret_key = '8B94BFE66DE11E295D925B73A4DD6'
```

```
    app.config['SESSION_TYPE'] = 'filesystem'

    app.run(debug=True, port=8000)
```

Inferer.py

In [ ]:

```
import numpy as np
import tensorflow as tf
import os

def loadModel(modelPath: str) -> tf.keras.models.Model | bool:
    """
    Loads a deep learning model using Keras.
    :param modelPath: The filepath of the saved model.
    :return: A saved model object ready for inference.
    :returns: False if the path does not exist or does not contain a model.
    """
    if not os.path.exists(modelPath):
        return False
    try:
        return tf.keras.models.load_model(modelPath)
    except OSError:
        return False

class Inferer:
    """
    An Inferer object used to make predictions
    """
    def __init__(self, modelPath: str):
        """
        The constructor for Inferer.
        :param modelPath: The filepath to saved model.
        """
        self.model = loadModel(modelPath)
        if self.model:
            self.validModel = True
            self.input_shape = tuple(self.model.input.shape[1:])
        else:
            self.validModel = False

    def predict(self, sample:tf.constant) -> (np.ndarray, bool):
        """
        Makes a prediction using the model.
        :param sample: A spectrogram or batch of spectrograms.
        :return: A numpy array of confidence values.
        :returns: A valid flag for error prevention.
        """
        try:
            result = self.model.predict(sample)
            return result, True
        except ValueError as error:
            return error, False
```

Main.py

In [ ]:

```
from SpectrogramGenerator import SpectrogramGenerator
from MidiGenerator import MidiGenerator
from music21.converter import parse
from Inferer import Inferer
import tensorflow as tf
import os


def loadInferer(path=r"model") -> Inferer | bool:
    """
    Loads and validates the deep learning model.
    :param path: The path of the saved model.
    :return: A custom Inferer object.
    :returns: False if the model is invalid.
    """
    inferer = Inferer(path)
    if not inferer.validModel:
        return False
    return inferer


def getInputFile() -> str | bool:
    """
    Finds the most recently uploaded file by the user
    :return: The filepath to be inferred.
    :returns: False if no suitable files can be found.
    """
    AllowedFiles = ['.wav', '.mp3', '.aac', 'ogg']
    audioDirectory = r'variables'
    files = [file for file in os.listdir(audioDirectory) if os.path.splitext(file)[
    try:
        sampleFilepath = os.path.join(audioDirectory, files[-1])
    except IndexError:
        return False
    return os.path.abspath(sampleFilepath)

def prepareData(path: str, input_shape: tuple) -> (tf.constant, dict, bool):
    """
    Generate spectrograms using a generator object.
    :param path: The serverside filepath of the audio recording.
    :param input_shape: The shape of the first layer of the deep learning model.
    :return: A 3-rank tensor of spectrograms.
    :return: A dictionary of the files' metadata.
    :return: A flag to determine if the function was successful or not.
    """
    generator = SpectrogramGenerator(path, input_shape)
    if generator.corrupt:
        return None, None, False
    samples, validSamples = generator.loadSample()
    if not validSamples:
        return None, None, False
    os.remove(path)
    spectrograms = [generator.getSpectrogram(sample) for sample in samples]
    tags = generator.getTags()
    return tf.stack(spectrograms), tags, True
```

```
def main():
    """
    Performs inference on the file uploaded to the server by the user.
    :return: A string with an error message if one is applicable.
    """

    """
    Load Model
    """
    inferer = loadInferer()
    if not inferer:
        return "Unable to load model."
    image_shape = inferer.input_shape

    """
    Load Audio Recording
    """
    samplePath = getInputFile()
    if not samplePath:
        return None

    """
    Generate Spectrograms
    """
    spectrograms, tags, validSpectrograms = prepareData(samplePath, image_shape)
    if not validSpectrograms:
        return "Could not read file: Corrupted!"

    """
    Make Predictions and Convert to MIDI
    """
    predictions, valid = inferer.predict(spectrograms)
    if not valid:
        return "Invalid model input shape"
    midi = MidiGenerator(tags)
    notes = midi.filterPredictions(predictions)
    midiFile = midi.generateMidi(notes)
    midiPath = os.path.join(r'variables', f"{tags['title']}.mid")
    with open(midiPath, 'wb') as file:
        midiFile.writeFile(file)

    """
    Visualise MIDI Through Lilypond
    """
    score = parse(midiPath)
    score.metadata.title = tags['title']
    score.metadata.composer = ""
    scorePath = os.path.join(r'static', 'score.pdf')
    score.write('musicxml.pdf', fp=scorePath)

    return None

if __name__ == '__main__':
    """
    The entry point for the Musical Notation Generator Application
```

```
    """
    main()
```

MidiGenerator.py

In [ ]:

```
import midiutil
from midiutil.MidiFile import MIDIFile
from collections import defaultdict
from itertools import chain
from math import log2
import numpy as np

def rle(bar: list, loops: int) -> list:
    """
    A recursive implementation of run-length encoding. Used to convert note positio
    :param bar: The positions within a bar of a single pitch.
    :param loops: The number of interactions left until all positions have been che
    :return: A list of note durations.
    """
    if loops == 0:
        return bar
    for i, v in enumerate(bar[:-1]):
        if v != 0:
            bar[i] = bar[i] + bar[i + 1]
            bar[i + 1] = 0
    return rle(bar, loops-1)


def inferBar(bar: list) -> dict:
    """
    Generates midi signals for a single bar of music
    :param bar: A single bars worth of pitches
    :return: A dictionary of pitches and when they occur in the bar.
    """
    notes = set(chain.from_iterable(bar))
    result = defaultdict(list)
    for beat in bar:
        for note in notes:
            if note in beat:
                result[note].append(1)
            else:
                result[note].append(0)

    for key, value in result.items():
        result[key] = rle(value, len(bar))
    return result

class MidiGenerator:
    """
    An object which creates midi objects from model predictions.
    """
    def __init__(self, tags: dict, threshold=0.6):
        """
        The default constructor.
        :param tags: The other features of the file such as tempo, time signature e
```

```
        :param threshold: The minimum confidence the model should have to approve a
        """
        self.threshold = threshold
        self.title = tags['title']
        self.tempo = tags['tempo']
        self.time_signature = tags['time_signature']
        self.division = 2**tags['note_values']

    def filterPredictions(self, predictions: np.ndarray) -> list:
        """
        Converts the inference results into pitches.
        :param predictions: The output from the model.
        :return: A list of pitches for every note.
        """
        return [np.argwhere(prediction > self.threshold).flatten().tolist() for pre


    def generateMidi(self, score: list) -> midiutil.MIDIFile:
        """
        Generates a midifile from a list of pitch values.
        :param score: The pitches for the whole song.
        :return: A midifile containing signal generated from inference.
        """
        mf = MIDIFile(1)

        mf.addTrackName(0, 0, self.title)
        mf.addTempo(0, 0, self.tempo)
        measureCount, noteType = list(map(int, list(self.time_signature)))
        mf.addTimeSignature(0, 0, measureCount, int(log2(noteType)), int(24/self.di

        for i in range(0, len(score), measureCount):
            bar = score[i:i+measureCount]
            notesInBar = inferBar(bar)
            for pitch, durations in notesInBar.items():
                for j, duration in enumerate(durations):
                    if duration != 0:
                        mf.addNote(0, 0, pitch, (i+j)/self.division, duration, 127)

        return mf
```

SpectrogramGenerator.py

In [ ]:

```
from scipy.io.wavfile import read
import tensorflow as tf
import numpy as np
import taglib
import os

class SpectrogramGenerator:
    """
    A custom object for pre-processing data before inference.
    """
    def __init__(self, filepath: str, model_shape: tuple):
        """
        The default constructor for the Spectrogram Generator object.
```

```
        :param filepath: The filepath for the audio file to be inferred.
        :param model_shape: A tuple with the dimensions for the first layer of the
        """
        self.filepath = filepath
        self.title = os.path.splitext(os.path.basename(filepath))[0]
        try:
            with taglib.File(filepath) as file:
                self.sampleRate = file.sampleRate
                tags = file.tags
                self.division = int(tags['NOTE_DIVISION'][0])
                self.tempo = float(tags['TEMPO'][0])
                self.timeSignature = tags['TIME_SIGNATURE'][0]
                self.targetFrameLength = model_shape[:-1]
                self.corrupt = False
        except OSError:
            self.corrupt = True
        except KeyError:
            self.corrupt = True

    def loadSample(self) -> (tf.constant, bool):
        """
        Loads an audio file and splits it into individual notes.
        :returns: A batch of tensors and a flag if the function was successful.
        """
        try:
            """
            Convert stereo to mono
            """
            wavfile = np.mean(read(self.filepath)[-1], axis=1)
        except ValueError:
            """
            Return False if file is corrupt.
            """
            return None, False
        samplesPerNote = self.getWindowSize()
        numSpectrograms = int(wavfile.shape[0] / samplesPerNote)
        samples = np.empty((numSpectrograms, samplesPerNote))
        for i in range(numSpectrograms):
            samples[i] = wavfile[i*samplesPerNote:(i+1)*samplesPerNote]
        return tf.convert_to_tensor(samples), True

    def getWindowSize(self) -> int:
        """
        Calculates the number of samples in a single note.
        :return: The window size of each note.
        """
        durationOfBeat = 60/self.tempo
        samplesPerBeat = self.sampleRate * durationOfBeat
        return int(samplesPerBeat/(2 ** self.division))

    def getSpectrogram(self, sample: tf.constant) -> tf.constant:
        """
        Converts a tensor of audio signals into a spectrogram.
        :param sample: The data for a single note from the audio file
        :return: A spectrogram.
        """
```

```
        spectrogram = tf.signal.stft(sample, frame_length=256, frame_step=64)
        spectrogram = tf.abs(spectrogram[:self.targetFrameLength[1]])
        padding = [
            [0, self.targetFrameLength[1]-spectrogram.shape[0]],
            [0, self.targetFrameLength[0]-spectrogram.shape[1]]
        ]
        """
        Padding ensures that the spectrograms are compatible with the model.
        """
        spectrogram = tf.pad(spectrogram, padding, "CONSTANT")
        spectrogram = tf.transpose(spectrogram)
        return tf.expand_dims(spectrogram, axis=2)

    def getTags(self) -> dict:
        """
        Parses the metadata from an audio file.
        :return: A dictionary of the metadata tags and their values.
        """
        return {'title': self.title, 'tempo': self.tempo, 'note_values': self.divis
```

Utilities.py

In [ ]:

```
from werkzeug.utils import secure_filename
from werkzeug.datastructures import FileStorage
import shutil
import taglib
import os

def checkFileType(file: str) -> bool:
    """
    Checks if the file uploaded by the user is of a valid type.
    :param file: The filename of the users file.
    :return: A boolean for if the type is valid or not.
    """
    AllowedFiles = ['.wav', '.mp3','.aac', 'ogg']
    if os.path.splitext(file)[-1] in AllowedFiles:
        return True
    return False

def tagMetadata(filepath: str, tags: dict) -> bool:
    """
    Appends metadata to the file uploaded to the server.
    :param filepath: The serverside filepath.
    :param tags: The metadata tags and values to append.
    :return: returns True upon completion.
    """
    with taglib.File(filepath, save_on_exit=True) as file:
        for tag in tags:
            file.tags[tag] = tags[tag]
    return True

def setupFilesystem():
    """
    This function runs on server startup and clears the filesystem from the previou
    :return: None
```

```
    """
    try:
        os.remove(r'static\Score.pdf')
    except FileNotFoundError:
        pass
    shutil.copy(r'variables\Score.pdf', 'static')
    for file in os.listdir('variables'):
        if file != "Score.pdf":
            try:
                os.remove(rf'variables\{file}')
            except FileNotFoundError:
                pass
    return None

def getFilePath(file: FileStorage) -> str:
    """
    Generates a serverside filepath for a file uploaded by the user.
    :param file: The file in the http request.
    :return: The serverside filepath.
    """
    filepath = os.path.join('variables', secure_filename(file.filename))
    return filepath
```

Unit Tests

test_Inferer.py

In [ ]:

```
from unittest import TestCase
from Application import Inferer
import keras
import numpy as np

class TestInferer(TestCase):
    def setUp(self) :
        self.goodModel = r"Resources\ValidModel"
        self.badModel = r"Resources\InvalidModel"

        self.testObj = Inferer.Inferer(self.goodModel)

    def test_loadModel_Good(self):
        model = Inferer.loadModel(self.goodModel)
        self.assertTrue(isinstance(model, keras.Model))

    def test_loadModel_Bad(self):
        model = Inferer.loadModel(self.badModel)
        self.assertFalse(model)

    def test_loadModel_NotFound(self):
        model = Inferer.loadModel(r"Resources\InvalidDir")
        self.assertFalse(model)

    def test_constructor_good(self):
        inferer = Inferer.Inferer(self.goodModel)
        self.assertIsInstance(inferer, Inferer.Inferer)
```

```
    def test_constructor_goodValid(self):
        inferer = Inferer.Inferer(self.goodModel)
        self.assertTrue(inferer.validModel)

    def test_constructor_bad(self):
        inferer = Inferer.Inferer(self.badModel)
        self.assertFalse(inferer.validModel)

    def test_predict_good(self):
        spectrogram = np.random.uniform(0, 1, (1, 129, 153))
        result, valid = self.testObj.predict(spectrogram)
        self.assertIsInstance(result, np.ndarray)

    def test_predict_goodValid(self):
        spectrogram = np.random.uniform(0, 1, (1, 129, 153))
        result, valid = self.testObj.predict(spectrogram)
        self.assertTrue(valid)

    def test_predict_bad(self):
        spectrogram = np.random.uniform(0, 1, 1)
        result, valid = self.testObj.predict(spectrogram)
        self.assertIsInstance(result, ValueError)

    def test_predict_badValid(self):
        spectrogram = np.random.uniform(0, 1, 1)
        result, valid = self.testObj.predict(spectrogram)
        self.assertFalse(valid)
```

test_Main.py

In [ ]:

```
from unittest import TestCase
import tensorflow as tf
import Inferer
import shutil
import Main
import os

class TestMain(TestCase):
    def setUp(self) :
        self.goodModel = r"Resources\ValidModel"
        self.badModel = r"Resources\InvalidModel"

        self.testInferer = Main.loadInferer(self.goodModel)
        self.test_shape = self.testInferer.input_shape

        shutil.copy(r"Resources\GoodAudio.wav", r"Resources\Temp")

        self.goodTestFile = r"Resources\Temp\GoodAudio.wav"
        spectrograms, tags, valid = Main.prepareData(self.goodTestFile, self.test_s
        self.spectrograms = spectrograms
        self.tags = tags
        self.prepare_valid = valid

    def test_loadInferer_valid(self):
```

```
        inferer = Main.loadInferer(self.goodModel)
        self.assertIsInstance(inferer, Inferer.Inferer)

    def test_loadInferer_invalid(self):
        inferer = Main.loadInferer(self.badModel)
        self.assertFalse(inferer)

    def test_loadInferer_error(self):
        inferer = Main.loadInferer(r"Resources\InvalidDir")
        self.assertFalse(inferer)

    def test_inferer_shape(self):
        inferer = Main.loadInferer(self.goodModel)
        self.assertEqual((129, 153, 1), inferer.input_shape)

    def test_getInputFile(self):
        os.chdir("../Application")
        result = Main.getInputFile()
        os.chdir("../UnitTest")
        self.assertFalse(result)

    def test_prepareData_type(self):
        self.assertTrue(tf.is_tensor(self.spectrograms))

    def test_prepareData_spectrogramShape(self):
        expected = self.test_shape
        actual = self.spectrograms[0].shape
        self.assertEqual(expected, actual)

    def test_prepareData_tags_type(self):
        self.assertIsInstance(self.tags, dict)

    def test_prepareData_tags(self):
        expected = {'title': 'GoodAudio', 'tempo': 120.0, 'note_values': 0, 'time_s
        self.assertEqual(self.tags, expected)

    def test_prepareData_valid(self):
        self.assertTrue(self.prepare_valid)
```

test_MidiGenerator.py

In [ ]:

```
from unittest import TestCase
from Application import MidiGenerator
from midiutil.MidiFile import MIDIFile
import numpy as np

class TestMidiGenerator(TestCase):
    def setUp(self) :
        self.tags = {"title": "title", "tempo": 120, "time_signature": "44", "note_
        self.testObj = MidiGenerator.MidiGenerator(self.tags)
        self.predictions = np.array([[1, 0, 0, 0], [0, 1, 0, 0], [0.4, 0.3, 0.2, 0.

    def test_constructor_good(self):
        obj = MidiGenerator.MidiGenerator(self.tags)
        self.assertIsInstance(obj, MidiGenerator.MidiGenerator)
```

```
    def test_constructor_bad(self):
        self.assertRaises(KeyError, lambda: MidiGenerator.MidiGenerator({}))

    def test_note_division(self):
        self.assertEqual(self.testObj.division, 1)

    def test_filterPredictions_valid(self):
        actual = self.testObj.filterPredictions(self.predictions)
        expected = [[0],[1],[],[2]]
        self.assertEqual(actual, expected)

    def test_filterPredictions_empty(self):
        actual = self.testObj.filterPredictions(np.array((1,)))
        expected = [[]]
        self.assertEqual(actual, expected)

    def test_generateMidi_good(self):
        mf = self.testObj.generateMidi([[0], [1], [2], [3]])
        self.assertIsInstance(mf, MIDIFile)

    def test_generateMidi_empty(self):
        mf = self.testObj.generateMidi([[],[],[],[]])
        self.assertIsInstance(mf, MIDIFile)

    def test_inferBar_good(self):
        actual = MidiGenerator.inferBar([[1, 2],[2, 3],[3, 4],[4, 5]])
        expected = {1: [1, 0, 0, 0], 2: [2, 0, 0, 0], 3: [0, 2, 0, 0], 4: [0, 0, 2,
        self.assertEqual(actual, expected)

    def test_inferBar_empty(self):
        actual = MidiGenerator.inferBar([[],[],[],[]])
        expected = {}
        self.assertEqual(actual, expected)

    def test_inferBar_type(self):
        result = MidiGenerator.inferBar([[],[],[],[]])
        self.assertIsInstance(result, dict)

    def test_rle_good(self):
        actual = MidiGenerator.rle([1, 1, 1, 1], 4)
        expected = [4, 0, 0, 0]
        self.assertEqual(actual, expected)

    def test_rle_variation(self):
        actual = MidiGenerator.rle([0, 1, 1, 0], 4)
        expected = [0, 2, 0, 0]
        self.assertEqual(actual, expected)

    def test_rle_gap(self):
        actual = MidiGenerator.rle([1, 1, 0, 1], 4)
        expected = [2, 0, 0, 1]
        self.assertEqual(actual, expected)

    def test_rle_empty(self):
        actual = MidiGenerator.rle([], 4)
```

```
        expected = []
        self.assertEqual(actual, expected)

    def test_rle_type(self):
        result = MidiGenerator.rle([], 4)
        self.assertIsInstance(result, list)
```

test_SpectrogramGenerator.py

In [ ]:

```
from unittest import TestCase
import SpectrogramGenerator
import tensorflow as tf
import numpy as np
import os

class TestSpectrogramGenerator(TestCase):
    def setUp(self) :
        self.goodTestFile = r"Resources\GoodAudio.wav"
        self.badTestFile = r"Resources\NotAudio!.txt"
        self.corruptTestFile = r"Resources\CorruptAudio.wav"

        self.testObj = SpectrogramGenerator.SpectrogramGenerator(self.goodTestFile,

    def test_constructor_good(self):
        obj = SpectrogramGenerator.SpectrogramGenerator(self.goodTestFile, (1, 1))
        self.assertIsInstance(obj, SpectrogramGenerator.SpectrogramGenerator)

    def test_constructor_bad(self):
        obj = SpectrogramGenerator.SpectrogramGenerator(self.badTestFile, (1, 1))
        self.assertTrue(obj.corrupt)

    def test_constructor_corrupt(self):
        obj = SpectrogramGenerator.SpectrogramGenerator(self.corruptTestFile, (1, 1
        self.assertTrue(obj.corrupt)

    def test_filepath(self):
        self.assertEqual(self.testObj.filepath, self.goodTestFile)

    def test_filepath_exists(self):
        self.assertTrue(os.path.isfile(self.testObj.filepath))

    def test_title(self):
        self.assertEqual(self.testObj.title, "GoodAudio")

    def test_division(self):
        self.assertEqual(self.testObj.division, 0)

    def test_tempo(self):
        self.assertEqual(self.testObj.tempo, 120.0)

    def test_timeSignature(self):
        self.assertEqual(self.testObj.timeSignature, '44')

    def test_targetFrameLength(self):
        self.assertEqual(self.testObj.targetFrameLength, (150, 150))
```

```
    def test_getTags_good(self):
        expected = {'title': 'GoodAudio', 'tempo': 120.0, 'note_values': 0, 'time_s
        actual = self.testObj.getTags()
        self.assertEqual(expected, actual)

    def test_getSpectrogram_shape(self):
        sample = np.random.uniform(-44100, 44100, (768000,))
        spectrogram = self.testObj.getSpectrogram(sample)
        self.assertEqual(spectrogram.shape, (150, 150, 1))

    def test_getSpectrogram_smallerDivision(self):
        sample = np.random.uniform(-44100, 44100, (768000,))
        self.testObj.division = 2
        spectrogram = self.testObj.getSpectrogram(sample)
        self.assertEqual(spectrogram.shape, (150, 150, 1))

    def test_getSpectrogram_type(self):
        sample = np.random.uniform(-44100, 44100, (768000,))
        spectrogram = self.testObj.getSpectrogram(sample)
        self.assertTrue(tf.is_tensor(spectrogram))
```

test_Utilities.py

In [ ]:

```
from unittest import TestCase
from Application import Utilities
from werkzeug.datastructures import FileStorage
import os

class TestUtilities(TestCase):
    def setUp(self) :
        self.goodTestFile = r"Resources\GoodAudio.wav"
        self.badTestFile = r"Resources\NotAudio!.txt"

        with open(self.goodTestFile, 'rb') as f:
            self.goodFile = FileStorage(f)

        with open(self.badTestFile, 'rb') as f:
            self.badFile = FileStorage(f)

    def test_checkFileType_Good(self):
        result = Utilities.checkFileType(self.goodTestFile)
        self.assertTrue(result)

    def test_checkFileType_Bad(self):
        result = Utilities.checkFileType(self.badTestFile)
        self.assertFalse(result)

    def test_getFilePath_SafeFileName(self):
        expected = r"variables\Resources_GoodAudio.wav"
        actual = Utilities.getFilePath(self.goodFile)
        self.assertEqual(expected, actual)

    def test_getFilePath_NotSafeFileName(self):
        expected = r"variables\Resources_NotAudio.txt"
```

```
        actual = Utilities.getFilePath(self.badFile)
        self.assertEqual(expected, actual)

    def test_setupFilesystem_static(self):
        os.chdir(r"..\Application")
        Utilities.setupFilesystem()
        os.chdir(r"..\UnitTest")
        result = os.listdir(r"..\Application\static")
        self.assertTrue("Score.pdf" in result)

    def test_setupFilesystem_variables(self):
        os.chdir(r"..\Application")
        Utilities.setupFilesystem()
        os.chdir(r"..\UnitTest")
        expected = ["Score.pdf"]
        actual = os.listdir(r"..\Application\variables")
        self.assertEqual(expected, actual)
```

Musical Notation Generator

Comparison Of Different Models

~Adam McWilliams

This notebook loads and tests each of the models in a given directory

In [1]:
```
from glob import glob
import tensorflow as tf
import numpy as np
import os
```

In [2]:
```
path = r"models"
models = glob(os.path.join(path, "*"))
models.sort()
```

In [3]:
```
testData = r"E:\QUB\Stage3\CSC3002-Project\nsynth-data\nsynth-test\GeneratedDataset
dataset = tf.data.Dataset.load(testData)
```

In [4]:
```
def testModel(modelPath, dataset):
    model = tf.keras.models.load_model(modelPath)
    print(os.path.basename(modelPath))
    model.summary()
    print('\n')

    predicitons = model.predict(dataset.batch(128), verbose=0)
    _, labels = list(zip(*dataset))

    accuracyTest = [labels[i].numpy() == np.argmax(p) for i, p in enumerate(predici
    return round((accuracyTest.count(True)/len(accuracyTest))*100, 2)
```

In [5]:
```
accuracies = [testModel(model, dataset) for model in models]
```

CPU_Batch32_Epoch5
Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_2 (Conv2D)	(None, 127, 151, 256)	2560
activation_3 (Activation)	(None, 127, 151, 256)	0
max_pooling2d_2 (MaxPooling 2D)	(None, 63, 75, 256)	0
conv2d_3 (Conv2D)	(None, 61, 73, 256)	590080
activation_4 (Activation)	(None, 61, 73, 256)	0
max_pooling2d_3 (MaxPooling 2D)	(None, 30, 36, 256)	0
flatten_1 (Flatten)	(None, 276480)	0
dense_1 (Dense)	(None, 120)	33177720
activation_5 (Activation)	(None, 120)	0

Total params: 33,770,360
Trainable params: 33,770,360
Non-trainable params: 0

GPU_B32_E15
Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 127, 151, 256)	2560
activation (Activation)	(None, 127, 151, 256)	0
max_pooling2d (MaxPooling2D)	(None, 63, 75, 256)	0
conv2d_1 (Conv2D)	(None, 61, 73, 256)	590080
activation_1 (Activation)	(None, 61, 73, 256)	0
max_pooling2d_1 (MaxPooling 2D)	(None, 30, 36, 256)	0
flatten (Flatten)	(None, 276480)	0
dense (Dense)	(None, 127)	35113087
activation_2 (Activation)	(None, 127)	0

```
=================================================================
Total params: 35,705,727
Trainable params: 35,705,727
Non-trainable params: 0
_________________________________________________________________


GPU_B32_E15_OptimiserA
Model: "sequential_1"
_________________________________________________________________
 Layer (type)                 Output Shape              Param #
=================================================================
 conv2d_2 (Conv2D)            (None, 127, 151, 256)     2560

 activation_3 (Activation)    (None, 127, 151, 256)     0

 max_pooling2d_2 (MaxPooling  (None, 63, 75, 256)      0
 2D)

 conv2d_3 (Conv2D)            (None, 61, 73, 256)       590080

 activation_4 (Activation)    (None, 61, 73, 256)       0

 max_pooling2d_3 (MaxPooling  (None, 30, 36, 256)      0
 2D)

 flatten_1 (Flatten)          (None, 276480)            0

 dense_1 (Dense)              (None, 127)               35113087

 activation_5 (Activation)    (None, 127)               0

=================================================================
Total params: 35,705,727
Trainable params: 35,705,727
Non-trainable params: 0
_________________________________________________________________


GPU_B32_E5
Model: "sequential_1"
_________________________________________________________________
 Layer (type)                 Output Shape              Param #
=================================================================
 conv2d_2 (Conv2D)            (None, 127, 151, 256)     2560

 activation_3 (Activation)    (None, 127, 151, 256)     0

 max_pooling2d_2 (MaxPooling  (None, 63, 75, 256)      0
 2D)

 conv2d_3 (Conv2D)            (None, 61, 73, 256)       590080

 activation_4 (Activation)    (None, 61, 73, 256)       0
```

```
 max_pooling2d_3 (MaxPooling  (None, 30, 36, 256)      0
 2D)

 flatten_1 (Flatten)         (None, 276480)            0

 dense_1 (Dense)             (None, 127)               35113087

 activation_5 (Activation)   (None, 127)               0

=================================================================
Total params: 35,705,727
Trainable params: 35,705,727
Non-trainable params: 0
_________________________________________________________________
```

Model6

Model: "sequential_5"

Layer (type)	Output Shape	Param #
conv2d_13 (Conv2D)	(None, 65, 77, 32)	320
max_pooling2d_12 (MaxPooling2D)	(None, 32, 38, 32)	0
batch_normalization_11 (BatchNormalization)	(None, 32, 38, 32)	128
dropout_5 (Dropout)	(None, 32, 38, 32)	0
conv2d_14 (Conv2D)	(None, 32, 38, 64)	18496
max_pooling2d_13 (MaxPooling2D)	(None, 16, 19, 64)	0
batch_normalization_12 (BatchNormalization)	(None, 16, 19, 64)	256
dropout_6 (Dropout)	(None, 16, 19, 64)	0
conv2d_15 (Conv2D)	(None, 16, 19, 128)	73856
max_pooling2d_14 (MaxPooling2D)	(None, 8, 9, 128)	0
batch_normalization_13 (BatchNormalization)	(None, 8, 9, 128)	512
dropout_7 (Dropout)	(None, 8, 9, 128)	0
flatten_3 (Flatten)	(None, 9216)	0
dense_6 (Dense)	(None, 256)	2359552
dense_7 (Dense)	(None, 127)	32639

```
=================================================================
Total params: 2,485,759
Trainable params: 2,485,311
Non-trainable params: 448
_________________________________________________________________
```

Model8

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_4 (Conv2D)	(None, 65, 77, 32)	320
max_pooling2d_4 (MaxPooling 2D)	(None, 32, 38, 32)	0
batch_normalization (BatchN ormalization)	(None, 32, 38, 32)	128
dropout_4 (Dropout)	(None, 32, 38, 32)	0
conv2d_5 (Conv2D)	(None, 32, 38, 64)	18496
max_pooling2d_5 (MaxPooling 2D)	(None, 16, 19, 64)	0
batch_normalization_1 (Batc hNormalization)	(None, 16, 19, 64)	256
dropout_5 (Dropout)	(None, 16, 19, 64)	0
conv2d_6 (Conv2D)	(None, 16, 19, 128)	73856
max_pooling2d_6 (MaxPooling 2D)	(None, 8, 9, 128)	0
batch_normalization_2 (Batc hNormalization)	(None, 8, 9, 128)	512
dropout_6 (Dropout)	(None, 8, 9, 128)	0
flatten_1 (Flatten)	(None, 9216)	0
dense_3 (Dense)	(None, 256)	2359552
dense_4 (Dense)	(None, 127)	32639

```
=================================================================
Total params: 2,485,759
Trainable params: 2,485,311
Non-trainable params: 448
_________________________________________________________________
```

In [6]:
```
accuracies
```

Out[6]:
```
[65.8, 36.79, 80.37, 17.24, 89.84, 91.31]
```

Musical Notation Generator

Data Analysis and Pre-processing

~Adam McWilliams

This notebook is responsible for generating a Keras deep learning model which is able to detect the pitch of a given musical sample.

Section 0: Import Libraries

In [1]:
```
from scipy.io.wavfile import read
from collections import deque
from string import Template
from glob import glob
from tqdm import tqdm
import xml.etree.ElementTree as etree
import matplotlib.pyplot as plt
import tensorflow as tf
import numpy as np
import wave
import os
```

Section 1: Generate Annotations

This code is responsible for generating a .xml file which can reference the wave files downloaded from NSynth.
The path variable should point to a directory containing wave files with the format:
`instrumentName_instrumentSourceType_instrumentInt-pitch-velocity`

In [2]:
```
path = r'C:\Users\Adam\Documents\csc3002-40266924\Training\audio'
if os.path.isdir(path):
    foundAudioFiles = glob(os.path.join(path, '*'))
    fileNames = [os.path.basename(os.path.splitext(i)[0]) for i in foundAudioFiles]
    fileTypes = [os.path.splitext(i)[1] for i in foundAudioFiles]
else:
    print("Invalid Directory")
```

Each annotation within the xml file will follow this format. The data within the files is not stored here for better data management.

In [3]:
```
templateLabel = Template("""\t<$sample_name>
        <file>
            <filename>$filename</filename>
            <filetype>$filetype</filetype>
        </file>
        <sample>
```

```
            <sample_name>$sample_name</sample_name>
            <sample_rate>$sample_rate</sample_rate>
            <num_samples>$num_samples</num_samples>
        </sample>
        <features>
            <instrument_name>$instrument_name</instrument_name>
            <instrument_source>$instrument_source</instrument_source>
            <instrument>$instrument</instrument>
            <pitch>$pitch</pitch>
            <velocity>$velocity</velocity>
        </features>
    </$sample_name>\n""")
```

In [4]:
```
labelFileName = "labels"
outputFilePath = os.path.join(os.path.dirname(path), labelFileName)
if not outputFilePath.endswith('.xml'):
    outputFilePath = outputFilePath + '.xml'

if os.path.isfile(outputFilePath):
    print(f"File {outputFilePath}, already exists")
else:
    with open(outputFilePath, 'xt') as f:
        f.write('<root>\n')
```

This code will iterate through ever file in the path directory and generate an annotation for it and then append it to the labels xml file.

In [5]:
```
for i, file in enumerate(tqdm(foundAudioFiles)):
    if fileTypes[i] == '.wav':
        with wave.open(file, 'rb') as openFile:
            properties = openFile.getparams()

        if not fileNames[i].startswith('synth_lead'):
            instrumentName, instrumentSourceType, oth = fileNames[i].split('_')
            instrumentInt, pitch, velocity = oth.split('-')
        else:
            instrumentName, instrumentType, instrumentSourceType, oth = fileNames[i
            instrumentInt, pitch, velocity = oth.split('-')
            instrumentName = f"{instrumentName}_{instrumentType}"

        thisSampleDict = {
            'filename': f"{fileNames[i]}{fileTypes[i]}",
            'sample_name': fileNames[i],
            'filetype': fileTypes[i],
            'instrument_name': instrumentName,
            'instrument_source': instrumentSourceType,
            'instrument': instrumentInt,
            'pitch': pitch,
            'velocity': velocity,
            'sample_rate': properties[2],
            'num_samples': properties[3]
        }
        with open(outputFilePath, 'at') as f:
            f.write(templateLabel.substitute(thisSampleDict))
```

```
with open(outputFilePath, 'at') as f:
    f.write('</root>')
```

```
100%|██████████████████████████████████████████████████████████████████████████████|
4096/4096 [00:01<00:00, 2136.26it/s]
```

Section 2: Analysing dataset

This 'labels.xml' file can be investigated to gather insight into the datasets.

In [6]:
```
features = ['instrument','pitch']
featureSet = dict.fromkeys(features)
counts = {}

tree = etree.parse(outputFilePath)

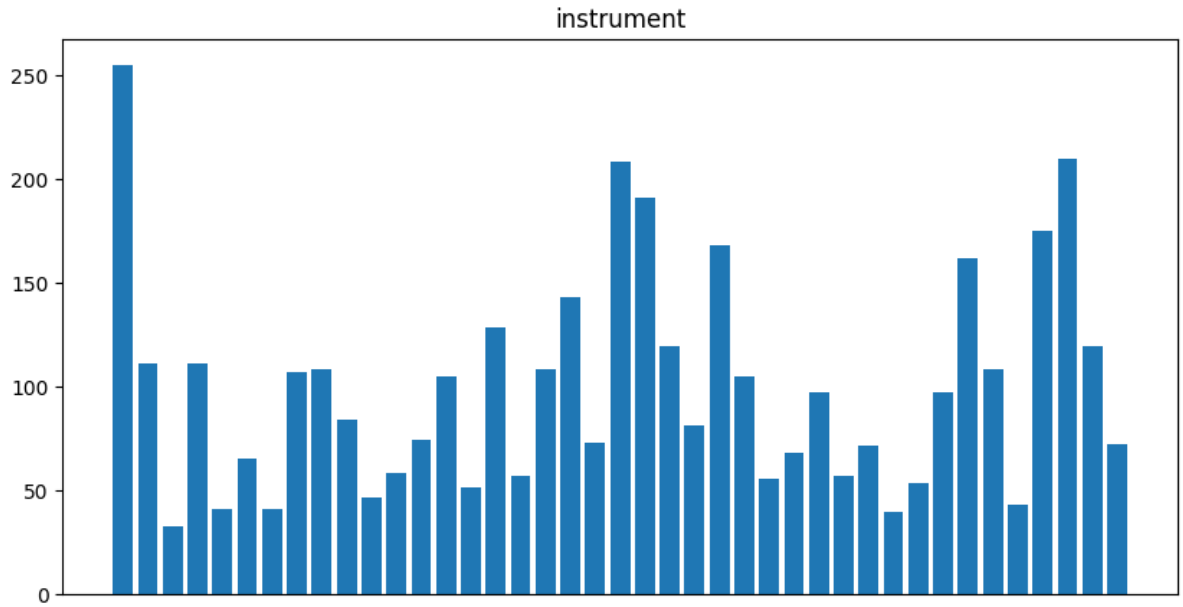
for feature in featureSet:
    thisFeatureList = [i.text for i in tree.iter(feature)]
    featureSet[feature] = thisFeatureList

    counts[feature] = {uniqueValue: thisFeatureList.count(uniqueValue) for uniqueVa
```

In [7]:
```
data = counts[features[0]]

plt.rcParams['figure.figsize'] = [10, 5]
plt.bar(range(len(data)), list(data.values()), align='center')
plt.xticks([])
plt.title(features[0])

plt.show()
```

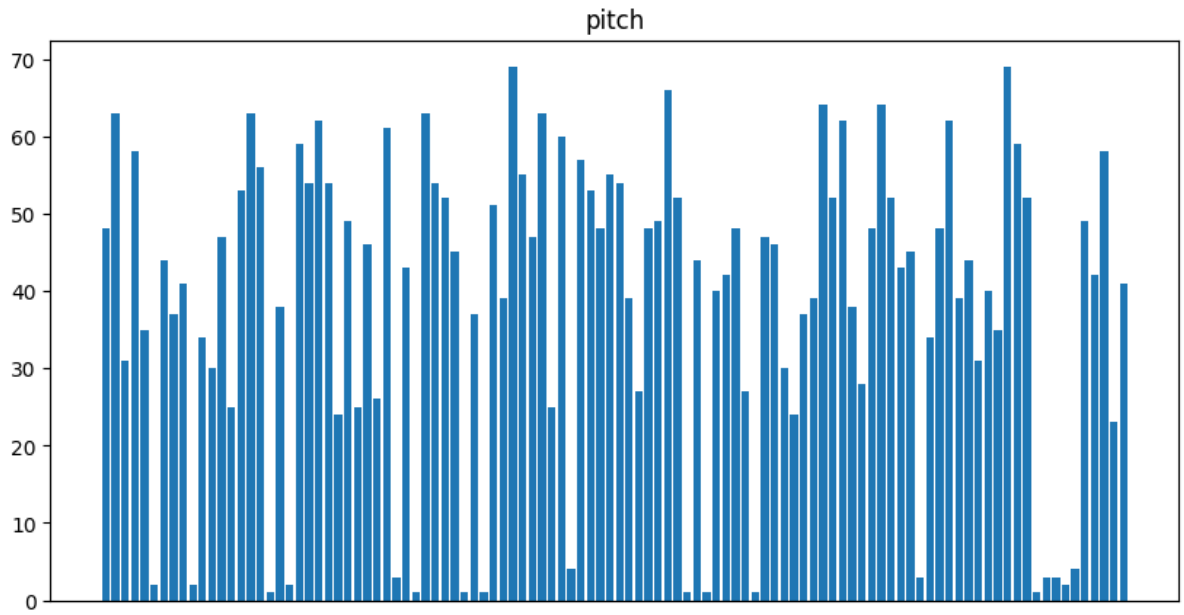


In [8]:
```
data = counts[features[1]]

plt.rcParams['figure.figsize'] = [10, 5]
plt.bar(range(len(data)), list(data.values()), align='center')
```

```
plt.xticks([])
plt.title(features[1])

plt.show()
```



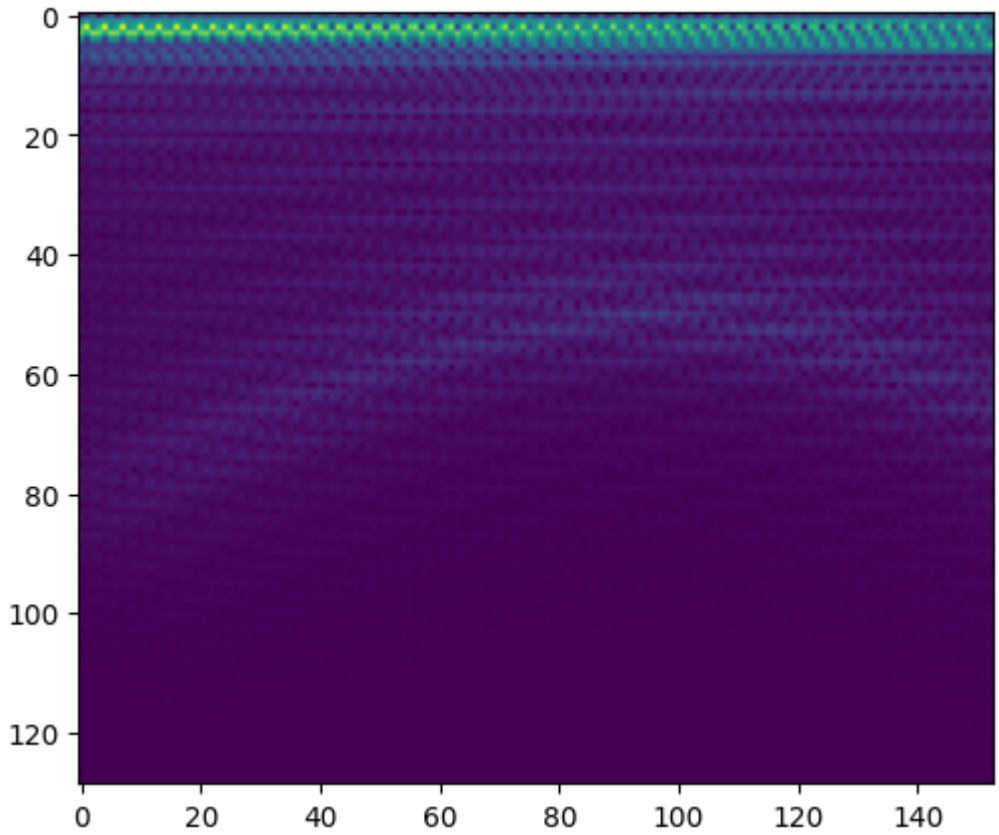
Section 3: Generating Spectrograms

These audio samples can now be used to investigate spectrogram creation.

In [9]:
```
def genSpec(samplePath):
    audioTensor = tf.convert_to_tensor(read(samplePath)[1])
    normalisedTensor = tf.squeeze(tf.cast(audioTensor, tf.float32) / 32768.0)
    spectrogram = tf.signal.stft(normalisedTensor[:10000], frame_length=256, frame_
    spectrogram = tf.abs(spectrogram)
    spectrogram = tf.expand_dims(spectrogram, axis=2)
    return tf.squeeze(spectrogram)
```

In [10]:
```
samplePath = foundAudioFiles[200]

plt.figure()
plt.imshow(tf.transpose(genSpec(samplePath)))
plt.show()
print(f"Resulting spectrogram is of size: {tf.transpose(genSpec(samplePath).shape)}
```




```
Resulting spectrogram is of size: [153 129] pixels
```

After generating a single spectrogram we are ready to convert the whole dataset

Section 4: Batch Spectrograms

This cell will convert all the files we have annotated into a tfrecord dataset.

In [11]:
```python
def generateSpectrograms(labelPath: str, sampleNames: list, labels=[], data=[]) ->
    tree = etree.parse(labelPath)
    for sample in tqdm(sampleNames):
        pitchStr = tree.find(sample).find('features').find('pitch').text
        try:
            thisPitch = int(pitchStr)
        except ValueError:
            return -1

        samplePath = os.path.join(path, f'{sample}.wav')
        try:
            thisSpectrogram = genSpec(samplePath)
        except FileNotFoundError:
            return -2

        thisSpectrogram = tf.transpose(thisSpectrogram)
        data.append(thisSpectrogram)
        labels.append(thisPitch)
    return data, labels
```

If the number of audio files take up a significant amount of memory they can be split into smaller batches to run on loss powerful machinery.

In [12]:
```
batches = 1
batchedSamples = np.array_split(fileNames, batches)

try:
    os.mkdir("batches")
except FileExistsError:
    pass

for i, batch in enumerate(batchedSamples):
    data, labels = generateSpectrograms(outputFilePath, batch)
    stackedData = tf.expand_dims(tf.stack(data), axis=-1)
    stackedLabels= tf.stack(labels)
    tfDataset = tf.data.Dataset.from_tensor_slices((stackedData, stackedLabels))
    datasetPath = os.path.join("batches", f'DataSet_{i*len(batch)}_to_{(i+1)*len(ba
    tfDataset.save(datasetPath)
```

```
100%|██████████████████████████████████████████████████████████████████
██| 4096/4096 [00:41<00:00, 98.47it/s]
```

Section 5: Combine Datasets

At this point the dataset has been made and is ready to train a model with.
However if you split the previous step into multiple batches this step will compile them together.

In [13]:
```
path = r'batches'
q = deque(glob(os.path.join(path, '*')))
```

In [14]:
```
with tf.device('/CPU:0'):
    ds = tf.data.Dataset.load(q.pop())

    while q:
        newDS = tf.data.Dataset.load(q.pop())

        images, labels = tuple(zip(*newDS))
        images = tf.expand_dims(tf.stack(images), axis=-1)
        newDS = tf.data.Dataset.from_tensor_slices((images, tf.stack(labels)))

        ds = ds.concatenate(newDS)

ds.save("Compiled_Dataset")
```

Musical Notation Generator

Deep Learning Model Definition and Training

~Adam McWilliams

This notebook is responsible for generating a Keras deep learning model which is able to detect the pitch of a given musical sample.

Section 0: Import Libraries

In [1]:
```
from glob import glob
import tensorflow as tf
import matplotlib.pyplot as plt
import numpy as np
import logging
import os
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Flatten, Input
from tensorflow.keras.layers import Conv2D, MaxPooling2D
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import TensorBoard
```

Throughout this notebook there are several techniques used to improve the efficiency when training the deep learning model.
The first of these techniques can be seen here by using TensorFlow to find available devices to place data structures onto.

In [2]:
```
tf.get_logger().setLevel(logging.ERROR)

gpus = tf.config.list_logical_devices('GPU')
if gpus:
    strategy = tf.distribute.MirroredStrategy(gpus, cross_device_ops=tf.distribute.
else:
    strategy = tf.distribute.get_strategy()

devices = [tf.config.list_physical_devices('CPU'), tf.config.list_physical_devices(
print(f'Found {len(devices[0])} CPU(s) and {len(devices[1])} GPU(s)')
```

```
Found 1 CPU(s) and 2 GPU(s)
```

Section 1: Read TFRecord Dataset

The data that we have prepared earlier exists as a tensorflow dataset and is arranged as follows:
 image.png
There are one of these dataset for each of the test, train, validation datasets. Each of them

must be loaded before the model can be trained.
This cell will take a directory which contains each of the datasets. It will return the paths to the datasets it can find

In [3]:
```
upperDirectory: str = r'E:\QUB\Stage3\CSC3002-Project\nsynth-data'
datasetName: str = 'GeneratedDataset'
datasets: list = glob(os.path.join(upperDirectory, '*', datasetName))

[print(dataset) for dataset in datasets];
```

```
E:\QUB\Stage3\CSC3002-Project\nsynth-data\nsynth-test\GeneratedDataset
E:\QUB\Stage3\CSC3002-Project\nsynth-data\nsynth-valid\GeneratedDataset
E:\QUB\Stage3\CSC3002-Project\nsynth-data\nsynth-train\GeneratedDataset
```

This cell defines the loop which is used to load each of the datasets. It makes calls to other functions to validate each dataset as they are loaded.

In [17]:
```
def loadDataset(path:str) -> (tf.data.Dataset, bool):
    """
    Loads a tf dataset from a filepath.
    :param path: The filepath for the tfrecord file.
    :return: A tuple where the first element is the loaded dataset.
    :return: The second element is a flag for is the function was successful.
    """
    try:
        with tf.device('/CPU:0'):
            dataset = tf.data.Dataset.load(path)
    except FileNotFoundError:
        return None, False
    return dataset, True

def verifyDatasets(datasets: list) -> bool:
    """
    Determines if all the datasets in a list have been loaded correctly.
    :param datasets: A list of loaded datasets.
    :return: A boolean value for if the datasets are all valid or not.
    """
    result = [isinstance(dataset, tf.data.Dataset) for dataset in datasets]
    if result == [True, True, True]:
        return True
    return False

def loadDatasets(paths: list) -> (list, bool):
    """
    Iterates through directories and loads them as tf datasets.
    :param paths: A list of directories.
    :return: A list of datasets and a validation flag.
    """
    datasets = [None, None, None]
    for path in paths:
        if 'train' in path:
            datasets[0], flag = loadDataset(path)
            if not flag:
                return [], False
        elif 'test' in path:
```

```
            datasets[1], flag = loadDataset(path)
            if not flag:
                return [], False
        elif 'valid' in path:
            datasets[2], flag = loadDataset(path)
            if not flag:
                return [], False
        else:
            pass
    if not verifyDatasets(datasets):
        return [], False
    return datasets, True
```

Here the above functions are called and the datasets are loaded into: `trainData, testData and validData` and will tell the user if it is successful or not.

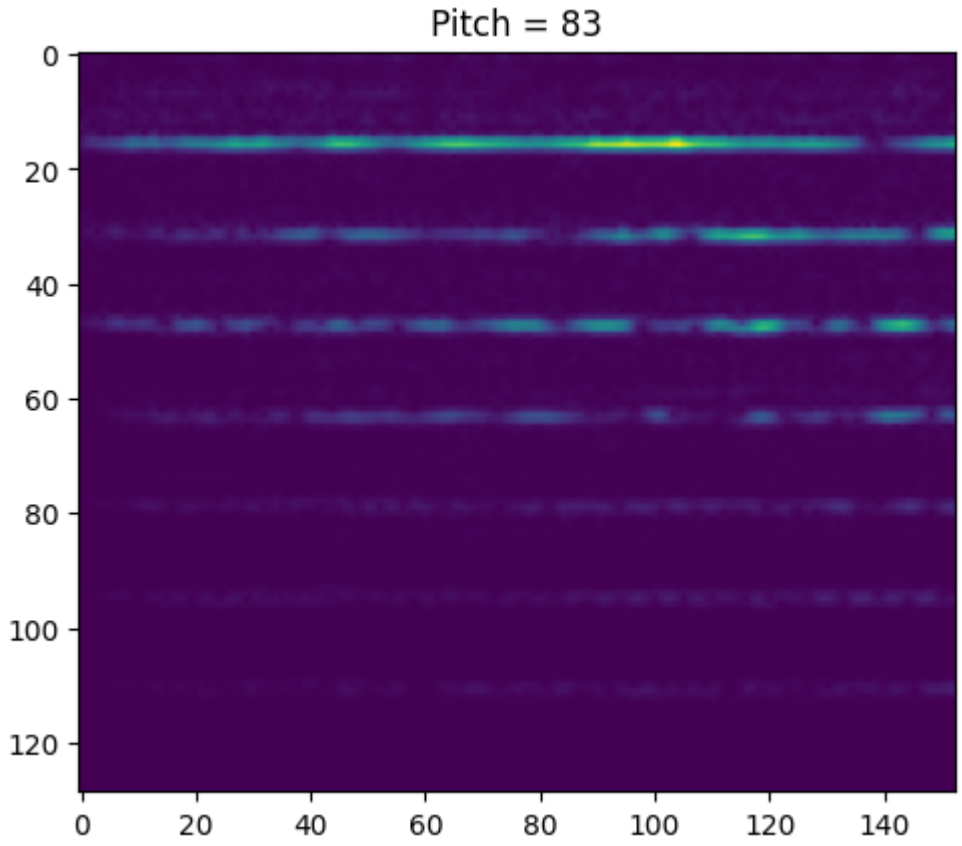
In [18]:
```
data, validLoading = loadDatasets(datasets)
if validLoading:
    trainData, testData, validData = data
    print('Successfully loaded datasets')
else:
    print('Unable to load datasets')
```

```
Successfully loaded datasets
```

Section 2: Visualise Test Data

To confirm the dataset contains usable data and to determine its size we will visualise one of the spectrograms. The size of this spectrogram is used later to define the input shape of the deep learning model.

In [22]:
```
sample, label = trainData.take(1).as_numpy_iterator().next()
plt.figure()
plt.imshow(sample)
plt.title(f"Pitch = {label}")
plt.show()
```



Section 3: Batch Dataset

The next cell will take the datasets we have loaded in memory and batch process them so that the model can train more efficiently. By re-batching the data it can be loaded into short term memory before it is fed into the model, drastically increasing training time. The prefetch method is also used to load even more of the data before so that it can perform even better.

`Using larger batch sizes is only possible on more powerful hardware.`

In [23]:
```
batchSize = 128
prefetchSize = int(batchSize/2)

trainData = trainData.batch(batchSize).prefetch(prefetchSize)
validData = validData.batch(batchSize).prefetch(prefetchSize)
```

Section 4: Define Model

This section is where the architecture of the deep learning model is defined. This code represents the 8th variation of the model.

You can see the convolutional layers that make up the model. There is also an advanced technique used here, `with stately.scope():` . This is using a distributed training strategy to spread the training evenly across all the GPUs evenly and uses the CPU for data management.

In [25]:
```
model_name = 'Model8'
tensorboard = TensorBoard(log_dir="logs/{}".format(model_name))
```

In [ ]:
```
with strategy.scope():
    model = Sequential()
    model.add(Input(shape=sample.shape))
    model.add(Dropout(0.2))
    model.add(Conv2D(64, (3,3), activation='relu'))
    model.add(MaxPooling2D(pool_size=(2,2)))
    model.add(Conv2D(128, (3,3), activation='relu'))
    model.add(MaxPooling2D(pool_size=(2,2)))
    model.add(Conv2D(128, (3,3), activation='relu'))
    model.add(MaxPooling2D(pool_size=(2,2)))
    model.add(Conv2D(256, (3,3), activation='relu'))
    model.add(MaxPooling2D(pool_size=(2,2)))
    model.add(Flatten())
    model.add(Dropout(0.5))
    model.add(Dense(512, activation='relu'))
    model.add(Dropout(0.5))
    model.add(Dense(256, activation='relu'))
    model.add(Dropout(0.5))
    model.add(Dense(128, activation='linear'))
```

Section 5: Compile Model

After defining the model it is now ready to be trained. To accomplish this we must first compile the model and define the training parameters.

In [ ]:
```
with strategy.scope():
    optimizer = Adam(learning_rate=0.001)
    model.compile(
        loss='sparse_categorical_crossentropy',
        optimizer=optimizer,
        metrics=['accuracy']
    )
```

In [ ]:
```
model.summary()
```

Section 6: Model Training

The model is now ready to be fit against the training dataset. During training the progress of the model can be viewed using tensorboard which can be accessed by running `tensorboard --logdir=logs/.` In a command prompt.

In [ ]:
```
model.fit(
    trainData,
    epochs=100,
    batch_size=128,
    validation_data=validData,
```

```
    callbacks=[tensorboard],
    verbose=0
)
```

The following are charts produced from Tensorboard data for `Model8` over the first 50 epochs. png

Section 7: Model Validation and Export

After the model is trained its final accuracy can be calculated against the test dataset.

In [117…
```
predictions = model.predict(testData.batch(128), verbose=0)
_, labels = list(zip(*testData))

accuracyTest = [labels[i].numpy() == np.argmax(p) for i, p in enumerate(predictions
acc = round((accuracyTest.count(True)/len(accuracyTest))*100, 2)
print(f'Total tests: {len(accuracyTest)}')
print(f'Accuracy: {acc}%')
```

```
Total tests: 4096
Accuracy: 91.31%
```

In [ ]:
```
model.save(model_name)
```